

Rapport de Stage de Fin d'Études

présenté par

Ayman EL MANNOUY

Stage effectué du 10/03/2025 au 05/09/2025 chez :

CS Group

Sujet du stage :

Extraction et structuration des connaissances à partir des enregistrements de conférence pour un système RAG

Tuteur de stage : **M. Jérôme CIUTI**

Co-tuteur académique : **M. Mohamed-Harith IBRAHIM**

La diffusion de ce rapport de stage a été approuvée par mon tuteur, Jérôme CIUTI, avant sa publication. Le sujet du stage ainsi que le contenu du rapport ne sont soumis à aucune restriction de confidentialité.

Remerciements

Je tiens à exprimer ma profonde gratitude à mon tuteur, Monsieur Jérôme CIUTI, pour son encadrement bienveillant, son partage de connaissances, et pour la valeur qu'il a su apporter au sujet de mon stage. Son accompagnement a grandement enrichi cette expérience.

Je remercie également mon co-tuteur Monsieur Mohamed-Harith IBRAHIM pour son suivi régulier et sa disponibilité tout au long de cette mission.

Un grand merci à mon entreprise CS GROUP pour la confiance qu'elle m'a accordée en me donnant accès à des ressources privées, parfois sensibles, afin de mener ce projet à bien dans les meilleures conditions.

Je tiens aussi à adresser un immense remerciement aux équipes ZAOG et Digital Alliance, avec qui j'ai partagé les locaux. Merci à Margaux CITERICI, Claire FRERY, Amandine LACOMBE, Lydia MESSAOUI, Alexandre MUSET, Alexandre RASCHIA, Johanna SALEWYN, Eya SMATI, Sébastien NIARFEIX, Selma BENABID, Eric BENHAMOU et Augustin VOISIN-MARRAS pour m'avoir accueilli comme un membre à part entière. Leur intégration chaleureuse, nos échanges quotidiens et les moments partagés en dehors du cadre professionnel — notamment lors de séances sportives et amicales — ont rendu cette expérience aussi enrichissante sur le plan humain que sur le plan technique.

Je remercie également mon collègue Idriss BENSOUDA, lui aussi stagiaire, avec qui j'ai construit ce projet dans une dynamique de collaboration fluide, marquée par une entraide mutuelle et une excellente communication.

S'il fallait choisir une expérience pour refermer mon livre académique, celle-ci en serait un excellent dernier chapitre.

* * *

Table des matières

Table des matières	4
1 Présentation de l'entreprise	7
1 Activité générale de l'entreprise	7
2 Domaines d'expertise	7
3 Positionnement et croissance	7
4 Intégration dans le groupe Sopra Steria	8
5 Présence géographique	8
6 Organisation de l'entreprise	8
7 Pôle Data and Process Intelligence	9
2 Présentation du sujet de stage	11
1 Projet Doc2Value	11
2 Origine du projet <i>Tech Event Expert</i>	11
3 Spécificités et différences avec les projets précédents	12
4 Ressources et collaboration	13
5 Conception du produit	13
6 Travail en équipe	15
7 Attendus techniques du stage	16
3 Travaux réalisés	19
1 Recherche bibliographique	19
1.1 Technologies de reconnaissance vocale automatique (ASR)	19
1.1.1 Problématique	19
1.1.2 Choix technique	20
1.2 Segmentation du texte	21
1.2.1 Problématique	21
1.2.2 Choix technique	22
1.3 Diarisation	22
2 Construction du pipeline	23
2.1 Processus d'intégration des nouveaux fichiers	23
2.2 Processus de pré-traitement	24
2.2.1 Algorithme d'unicité des fichiers	24
2.2.2 Normalisation du format des fichiers	25
2.2.3 Post pré-traitement	26
2.3 Processus de transcription automatique	26
2.4 Processus de diarisation	27

2.5	Processus de découpage	29
2.6	Post-traitement	31
3	Architecture de la base de données	32
3.1	Choix des bases de données	32
3.2	Schéma de données pour les transcriptions	33
3.3	Indexation et recherche	33
4	R&T exploratoire	34
4.1	Vérification de sécurité du format du fichier	34
4.2	Identification des interlocuteurs à l'aide de leur empreinte sonore . . .	35
4.3	Enrichissement d'identification des interlocuteurs (metadonnées exploitables dans les pages web descriptives des conférence)	35
4.4	Identification des interlocuteurs dans un enregistrement	36
4	Difficultés rencontrées et solutions	39
1	Prompt engineering	39
2	Limitations du service Groq	39
3	Limitations de centralisation de la base de données vectorielle	40
5	Conclusions et perspectives	41
1	Bilan du travail réalisé	41
1.1	Objectifs initiaux et objectifs atteints	41
1.2	Apports pour l'entreprise	41
2	Apports personnels	42
2.1	Compétences développées	42
2.2	Impact sur le projet professionnel	42
3	Perspectives d'évolution	43
3.1	Suites possibles du projet	43
3.2	Améliorations techniques envisageables	43
3.3	Evaluation numérique	44
	Table des figures	46
	Bibliographie	47
A	Annexes	53
1	Documentation du projet réalisé	54
2	Recherches bibliographiques faites en amont des décisions sur des choix techniques du projet	90
2.1	Technologies de reconnaissance vocale automatique (ASR)	91
2.1.1	Introduction	91
2.1.2	L'approche hybride classique	91
2.1.3	L'arrivée des modèles neuronaux	92
2.1.4	Comparaison hybride et <i>end-to-end</i>	92
2.2	Modèles de transcription	93
2.2.1	Comparaisons	93
2.2.2	Conclusion	94
2.3	Segmentation du texte pour un système RAG	95
2.3.1	Méthodes heuristiques (indépendantes du sens)	95

	2.3.2	Méthodes sémantiques (orientées cohérence)	98
	2.4	Introduction	103
3		Modèles récents de diarisation	103
	3.1	pyannote.audio	103
	3.2	NVIDIA NeMo	104
	3.3	EEND et DiaPer	104
	3.4	Comparaison des approches	104
	3.5	Conclusion	104

Présentation de l'entreprise

1 Activité générale de l'entreprise

CS GROUP est une entreprise française de taille intermédiaire (ETI), spécialisée dans la conception, l'intégration et l'exploitation de systèmes critiques intelligents et cyberprotégés. Elle intervient principalement dans des secteurs exigeants tels que la défense, l'espace, l'aéronautique, l'énergie, les transports, l'industrie 4.0 et les services financiers. Fondée en 1902 sous le nom de Compagnie des Signaux et d'Entreprises Électriques, elle a évolué au fil des décennies pour devenir CS Communication & Systèmes en 1999, puis CS GROUP en 2019. En 2023, l'entreprise a rejoint le groupe Sopra Steria, qui en détient désormais près de 94% du capital.

2 Domaines d'expertise

CS GROUP couvre l'ensemble de la chaîne de valeur des systèmes technologiques : du conseil à la conception, en passant par le développement, l'intégration, la maintenance et le support opérationnel. Son expertise s'articule autour de plusieurs axes majeurs : les systèmes de commandement et de surveillance, les communications vocales et tactiques, la cybersécurité (notamment via des solutions souveraines comme Prelude ou Trusty), les systèmes embarqués et industriels sécurisés, la data intelligence et l'intelligence artificielle, la digitalisation des processus industriels ainsi que la simulation.

L'entreprise se distingue particulièrement dans des domaines tels que la défense et la sécurité, avec des solutions anti-drones ou de surveillance maritime ; l'aéronautique, à travers des systèmes de radiocommunication sol-air ; et le spatial, avec la contribution à des segments sol de satellites ou à des systèmes comme EGNOS. Dans l'industrie 4.0, CS GROUP propose des solutions de modélisation, de simulation numérique, de cybersécurité industrielle et de digitalisation des procédés.

3 Positionnement et croissance

CS GROUP emploie entre 2200 et 2600 collaborateurs, principalement des ingénieurs et des experts dans leurs domaines. En 2022, son chiffre d'affaires avoisinait 273 millions d'euros, témoignant d'une croissance constante. L'entreprise a lancé un ambitieux plan stratégique intitulé Vision 2024, visant à renforcer sa présence dans les secteurs de la défense, du spatial et de l'industrie du futur. Ce plan prévoit une croissance tant organique qu'externe, en mettant l'accent sur l'innovation, l'exportation et les solutions souveraines.

Dans le cadre de cette stratégie, CS GROUP a également annoncé un plan de recrutement de 600 collaborateurs en 2022, dont 520 en France, pour soutenir son développement dans les régions d'Île-de-France, Occitanie et Provence-Alpes-Côte d'Azur.

4 Intégration dans le groupe Sopra Steria

L'intégration au sein de Sopra Steria permet à CS GROUP de renforcer ses capacités industrielles et financières. Ce rapprochement stratégique ouvre de nouvelles perspectives dans la réalisation de projets d'envergure, notamment dans les domaines du nucléaire, de la cybersécurité, du spatial et de la défense, en capitalisant sur les synergies techniques et commerciales des deux entités. Il conforte également la position de CS GROUP comme acteur clé des technologies souveraines et sécurisées en Europe.

5 Présence géographique

CS GROUP est largement implantée en France, avec des sites dans les principales régions économiques (Paris, Lille, Bordeaux, Toulouse, Aix-en-Provence, Toulon, Lyon, Grenoble, etc.). À l'international, elle dispose de filiales ou de représentations en Allemagne, Roumanie, Inde, Canada, États-Unis et Émirats arabes unis, lui permettant d'accompagner ses clients sur des projets globaux.

Pourtant, malgré sa présence internationale CS Group reste fortement ancré économiquement en France. En 2021, près des quatre cinquièmes de son chiffre d'affaires (79,3%) étaient générés dans l'Hexagone, tandis que l'international représentait environ 20,7% répartis entre Europe (14,4%) et autres régions du monde (6,3%)

6 Organisation de l'entreprise

À la tête de l'entreprise, on trouve le Président du Conseil d'Administration, Yazid Sabeg, chargé de définir les grandes orientations stratégiques. Il travaille en collaboration avec le Directeur Général, Pierre Lopez, qui supervise la gestion quotidienne de l'entreprise, pilote les équipes opérationnelles et s'assure de la bonne exécution des projets dans le respect des objectifs fixés.

Sous l'autorité du Directeur Général, CS GROUP est divisée en plusieurs Business Units (BUs), chacune centrée sur un secteur stratégique clé, tels que la Défense & Sécurité, l'Espace, l'Aéronautique, l'Industrie 4.0, les Transports ou encore l'Énergie. Ces BUs fonctionnent comme des entités semi-autonomes, dirigées par des Directeurs de BU qui sont responsables de la gestion opérationnelle, du pilotage commercial et du suivi technique des projets. Pour être au plus près des besoins des clients, chaque BU est elle-même structurée en plusieurs agences réparties géographiquement, implantées dans différentes régions comme Paris, Toulouse, Lyon, Aix-en-Provence, Toulon, Bordeaux, Lille ou Grenoble. Ces agences locales assurent une proximité terrain, facilitant les échanges, la réactivité, et l'adaptation aux spécificités régionales. Chaque BU regroupe des équipes pluridisciplinaires composées d'ingénieurs, de développeurs, d'analystes et d'experts métier, qui collaborent pour concevoir, intégrer et maintenir des solutions technologiques complexes adaptées aux secteurs les plus exigeants.

En parallèle, CS GROUP s'appuie sur un certain nombre de directions fonctionnelles transverses, comme la direction financière, la direction des ressources humaines, la direction marketing et communication, ou encore la direction des systèmes d'information. Ces départements assurent un soutien homogène à toutes les BUs, en veillant au respect des procédures internes, à la cohérence stratégique et à l'optimisation des ressources humaines, technologiques et financières.

Pour renforcer la transversalité et mutualiser les compétences à haut niveau, l'entreprise a également mis en place un centre d'expertises technologiques nommé *CS data lab*, regroupant plus de 200 collaborateurs spécialisés. Ce centre accompagne l'ensemble des unités opérationnelles dans des différents domaines tels que la cybersécurité, la simulation, l'intelligence artificielle, l'ingénierie système, ou encore les technologies spatiales et militaires. Il assure aussi une veille technologique constante et participe aux efforts de recherche et développement (R&D) du groupe.

7 Pôle Data and Process Intelligence

Le Pôle *Data and Process Intelligence* est une sous-business unit intégrée à la *Business Unit Industrie*. Sa mission principale est la valorisation et le traitement des données, notamment celles collectées par les clients mais souvent peu exploitées. En transformant ces données brutes en informations à forte valeur ajoutée, le pôle aide ses clients à améliorer leur performance, optimiser leurs processus et prendre des décisions plus éclairées. Il intervient aussi bien sur l'intégration de solutions existantes que sur le développement de nouvelles technologies adaptées aux besoins spécifiques des projets.

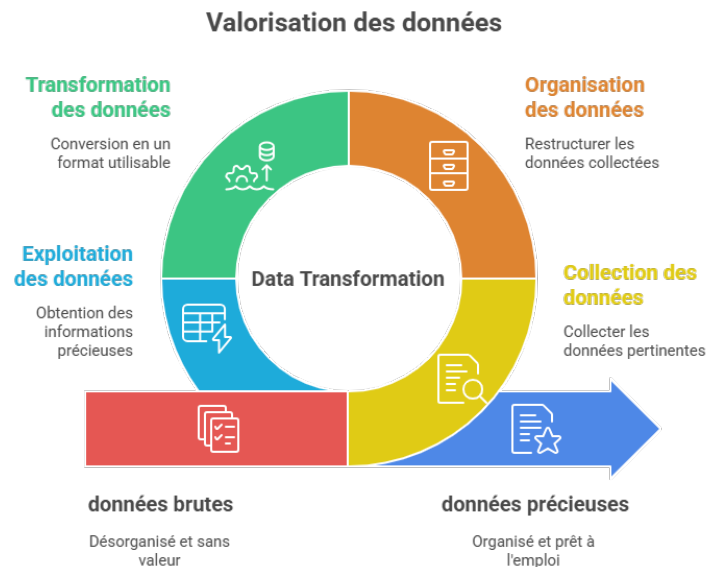


Fig. 1.1 Graphique qui présente l'activité du Pôle Data and Process Intelligence

Le pôle adresse en 2025 trois thématiques différentes au travers de trois projets phares :

- **Zero Aircraft On Ground (ZAOG)** : Projet dédié au développement d’algorithmes de maintenance prédictive et de *health monitoring* qui viennent enrichir une plateforme commerciale à destination de compagnies aériennes souhaitant bénéficier d’un service digital de surveillance et de management de leur flotte.
- **Digital Alliance (DA)** : Projet qui a pour but d’accompagner des experts métier et systèmes pour modéliser et déployer des algorithmes de maintenance prédictive et ainsi les mettre à disposition dans la même plateforme commerciale que celle de ZAOG. Au niveau des clients finaux (les compagnies aériennes), ils ont à leur disposition des modèles de maintenance prédictive sans besoin d’être en contact direct avec les experts métier[2].
- **SYM** : Projet dédié au croisement de la science des données et de la stratégie des marchés (donc la stratégie d’entreprise). L’intérêt majeur est porté sur les marchés liés à l’exploitation des avions commerciaux. Thématique aussi de gestion de crise via les capacités de modélisation selon le scénario émergent. Dans ce cadre CS participe activement à la démarche en amenant une expertise en manipulation, compréhension des données puis en création de modèles prédictif à partir de ces données.

Mon stage se déroule au sein de ce pôle, dans le cadre d’un projet de recherche indépendant, qui ne relève directement d’aucune des trois équipes. Mon tuteur appartient à l’équipe SYM, ce qui m’a permis de bénéficier de son expertise en gestion et pilotage de projet. Par ailleurs, j’ai entretenu une communication fluide avec l’équipe Digital Alliance, qui nous a apporté un soutien précieux sur certaines parties techniques du projet.

Présentation du sujet de stage

1 Projet Doc2Value

Chronologiquement, Doc2Value est un sujet de stage proposé en 2023. L'objectif initial était de développer un démonstrateur capable d'extraire des données critiques à partir de documents PDF, en s'appuyant sur des méthodes de traitement automatique du langage naturel (NLP). C'est de cette finalité que provient le nom Doc2Value (document to value).

DocValue s'est élargi dans les années qui suivent pour devenir une thématique globale qui a évolué dans sa stratégie avec l'avancée des techniques de traitement de langage naturel. Avec en 2024 la première exploitation d'un système RAG nommé MyCSGroupAssistant. Celui-ci visait à concevoir un assistant basé sur un grand modèle de langage (LLM), exploitant lui aussi des documents PDF(texte et texte dans image) pour répondre aux questions des collaborateurs de CS en lien avec leurs problématiques métiers.

En 2025, mon sujet de stage, intitulé Tech Event Expert, a vu le jour. Il s'intéresse à la valorisation des enregistrements vidéo et audio réalisés lors de salons et conférences, afin d'en extraire et d'exploiter les informations clés.

Ainsi, Doc2Value est un projet d'innovation et de recherche appliquée, tirant parti des technologies les plus récentes pour exploiter et valoriser des fichiers souvent sous-utilisés et dont la recherche d'information manuelle est chronophage.

Pour Tech Event Expert, deux stagiaires ont été attribués : Idriss Bensouda et moi-même. Cette dualité vise à créer une synergie et à pousser le projet encore plus loin.

2 Origine du projet *Tech Event Expert*

L'idée de *Tech Event Expert* est née à la suite de l'expression d'un besoin par un collaborateur d'Airbus (client de CS), qui évoquait les difficultés rencontrées face à la perte massive d'informations après la participation à des conférences.

Il est important de souligner que, chaque année, de nombreuses conférences se tiennent dans le monde sur des thématiques variées. Pour chaque événement jugé pertinent, une entreprise ne peut envoyer que quelques collaborateurs experts et non l'ensemble des spécialistes concernés, afin de limiter les coûts. Ces coûts incluent notamment les frais de déplacement, d'hébergement, les droits d'inscription à la conférence ainsi que l'indemnisation du collaborateur pendant sa mission.

À son retour, le participant a pour mission de partager les informations recueillies avec ses collègues. Ce transfert de connaissances se fait le plus souvent via des présentations ou de manière informelle (bouche-à-oreille). Ce mode de diffusion entraîne inévitablement une perte importante de contenu. Compte tenu des investissements financiers et de la valeur stratégique des informations échangées en conférence, il serait donc hautement bénéfique de disposer d'un moyen fiable et pérenne pour conserver et exploiter ces données.

Dans les faits, les participants enregistrent fréquemment, à l'aide de leur téléphone, des vidéos ou des fichiers audio des interventions. Ces enregistrements servent à rafraîchir leur mémoire ou à préparer un résumé à destination de leurs collègues. Cependant, dans la majorité des cas, ces fichiers restent inexploités et finissent oubliés dans un dossier.

C'est de cette problématique qu'est né le sujet de ce stage : **comment exploiter ces enregistrements pour en accroître la valeur informative et l'utilité ?**

Pour initier le projet, un collaborateur d'Airbus nous a transmis des enregistrements réalisés lors de la conférence WATS 2025 (*World Aviation Training Summit*), qui s'est tenue du 7 au 10 avril 2025. Les données fournies comprenaient :

- Les enregistrements audio capturés simplement avec son téléphone
- La page web officielle de la conférence, sauvegardée en local

Ce jeu de données a constitué la base des expérimentations menées pour tester les différentes fonctionnalités du système, comme cela sera détaillé dans la suite de ce rapport.

3 Spécificités et différences avec les projets précédents

À la différence des précédents projets de Doc2Value, Tech Event Expert représente un véritable défi et se distingue notamment de projets comme MyCSGroup par la nature des données traitées. Alors que les projets précédents s'appuyaient sur des documents PDF bien structurés — contenant paragraphes, titres et organisation logique —, Tech Event Expert travaille sur des fichiers audio et vidéo, c'est-à-dire sur des données non structurées.

Cette différence entraîne de nouveaux défis techniques. Par exemple, la qualité des fichiers n'est pas garantie : les enregistrements peuvent être bruités, incomplets ou de faible résolution, car souvent capturés simplement avec un téléphone. De plus, le contenu n'est pas nécessairement cohérent : il est fréquent que plusieurs intervenants se coupent la parole, rendant la transcription confuse et mêlant des propos parfois distincts, voire contradictoires.

Comparaison des défis de gestion des données

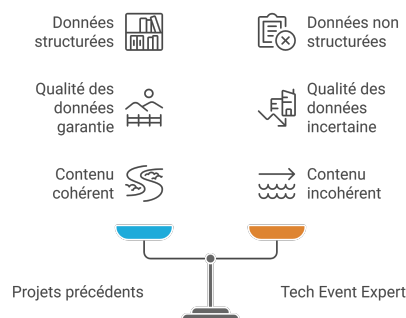


Fig. 2.1 Graphique qui explique les difficultés du projet TEX par rapport aux précédents projets Doc2Value

Il convient également de souligner que Tech Event Expert a été mené entièrement "from scratch", en toute indépendance vis-à-vis des projets précédents, tant sur le plan du code que de l'infrastructure. Les connaissances et l'expérience acquises lors des années antérieures ont toutefois été mises à profit pour guider et optimiser le développement du projet.

4 Ressources et collaboration

Pour mener à bien ce projet, il a été nécessaire de réaliser un grand nombre d'expérimentations tout au long du processus. Chaque expérience nécessitait des ressources matérielles spécifiques, comme des machines équipées de GPU pour l'exécution de certains modèles de *deep learning*, ou de petits serveurs CPU pour l'hébergement de bases de données. Afin de répondre à cette variabilité des besoins, nous avons utilisé les services AWS, qui nous ont permis de créer à la demande des instances de serveurs adaptées à chaque situation.

Les deux principaux services exploités étaient :

- **Amazon EC2** : serveurs virtuels modulables et dimensionnables selon les besoins en calcul ou en mémoire.
- **Amazon S3** : solution de stockage de données flexible, sans limite de quantité ou de taille.

En complément, nous avons également utilisé **Groq**[1], un service indépendant *serverless* permettant d'exécuter facilement des modèles de *deep learning* (LLM, modèles de vision) ainsi que le modèle de transcription *Whisper*, sans gestion d'infrastructure de notre part.

Nous disposions aussi d'un accès à un serveur GitLab privé (similaire à github), qui a facilité la collaboration et l'organisation du code au sein de l'équipe.

Nous avons également eu l'opportunité de participer aux ateliers du projet FOR, un programme de recherche piloté par IRT Saint Exupéry, couvrant un large éventail de thématiques en intelligence artificielle[25]. Ces ateliers réunissent des chercheurs en France et à l'international, qui y présentent leurs travaux. Ils nous ont permis de rester informés sur les avancées récentes et d'intégrer, lorsque pertinent, des idées innovantes issues de ces échanges dans notre propre projet.

5 Conception du produit

Le projet étant construit en binôme, il a fallu dès le départ nous organiser afin de développer la solution de manière efficace. La première étape a consisté à mener une réflexion approfondie sur la vision que nous avions du produit final. À partir de cette vision, nous avons identifié les grandes tâches à réaliser, puis nous les avons réparties entre Idriss et moi de façon à limiter au maximum notre dépendance mutuelle.

Nous avons ainsi rédigé un diagramme résumant le flux global des données et les différentes opérations qu'elles subissent, depuis leur entrée jusqu'à la sortie. Parallèlement, nous avons effectué des recherches bibliographiques pour mieux cerner les outils potentiellement utilisables, et envisager leur incorporation tout au long du projet.

La sortie du système prévoyait initialement trois possibilités :

- Un chatbot dont les connaissances reposent sur la base de données des enregistrements.
- La génération d'un rapport sur une thématique ou en réponse à une question liée à la base de données. Pour ce service, nous avons prévu d'utiliser la méthode classique du RAG : vectoriser la requête, la comparer à l'ensemble des vecteurs textuels de la base, et sélectionner les plus pertinents.
- Une méthode expérimentale (finalement abandonnée) qui consistait à demander directement à un LLM d'accepter ou rejeter un texte en fonction d'une requête. Bien que potentiellement plus lente, elle suscitait notre curiosité quant à sa qualité, mais elle ne s'alignait pas pleinement avec les objectifs du projet.

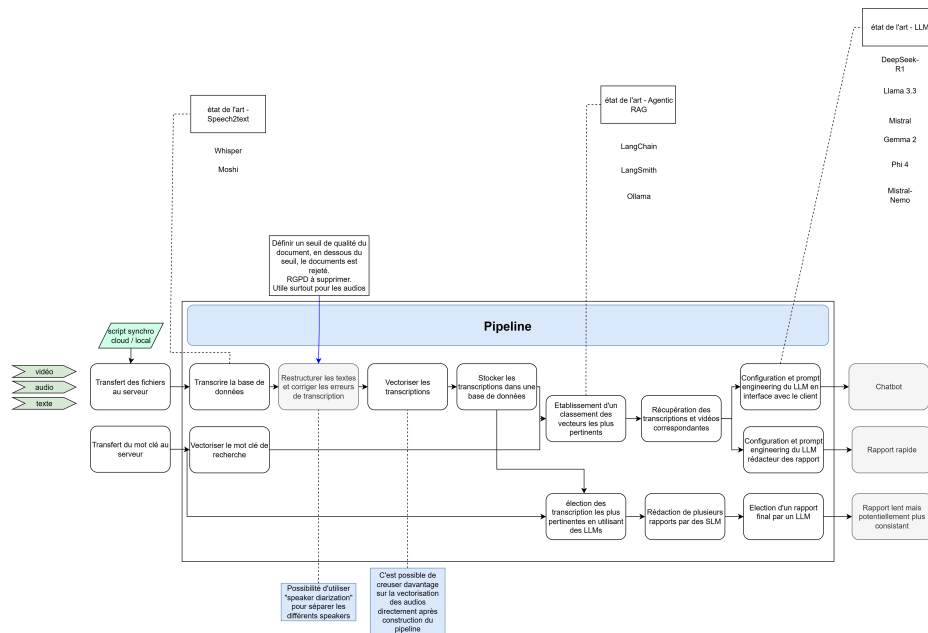


Fig. 2.2 Architecture initiale du flux des données du projet Tech Event Expert

Après cette première phase de réflexion, nous avons commencé à concevoir le fonctionnement détaillé du produit visé. En nous appuyant sur le diagramme précédent, nous avons précisé chaque partie, en veillant à structurer l'architecture en composants indépendants. L'objectif était double : garantir la maintenabilité du système et faciliter l'intégration et le déploiement sur des environnements potentiellement différents.

Chaque composant pouvait ainsi avoir ses propres dépendances, voire des prérequis matériels distincts. Par exemple, la gestion des fichiers ne nécessite pas de GPU, tandis que certaines tâches de *deep learning* en demandent impérativement pour avoir des délais de traitement raisonnables. Nous avons donc pris soin d'isoler complètement chaque module afin qu'ils puissent, si nécessaire, être déployés sur des machines distinctes. Cette approche permettait aussi d'éviter la création d'une seule application monolithique englobant l'ensemble des fonctionnalités.

Le flux final imaginé se décompose en quatre grandes étapes :

1. **Interface d'entrée des données** L'utilisateur envoie de nouveaux enregistrements au service (vidéos, audios et métadonnées associées). Ce composant se charge simplement de stocker ces données dans un répertoire S3, en attente du traitement ultérieur.
2. **Composant d'intégration des fichiers** Ce module intègre (ou écarte) certains fichiers en effectuant des vérifications de sécurité et en éliminant les doublons. Initialement, nous envisagions de transférer les fichiers dans leur format d'origine pour les étapes suivantes. Cependant, il est rapidement apparu qu'il serait plus judicieux de normaliser les formats dès cette étape. Nous avons donc décidé de convertir systématiquement toutes les vidéos et tous les audios en fichiers `.wav`. Ainsi, le reste du pipeline traitera toujours un format unique et standardisé. Les fichiers validés et convertis sont ensuite envoyés vers un autre répertoire S3.
3. **Composant de traitement des fichiers** C'est le cœur du flux. Il assure les transformations principales :
 - conversion des fichiers audio en texte.
 - segmentation de ce texte en morceaux.
 - construction d'une base de données textuelle et vectorielle à partir de ces segments.
4. **Rédacteur de rapports** Ce module reçoit une requête ou une question de l'utilisateur, recherche les informations pertinentes dans la base de données construite à l'étape précédente, puis génère un rapport synthétique sur le sujet demandé.

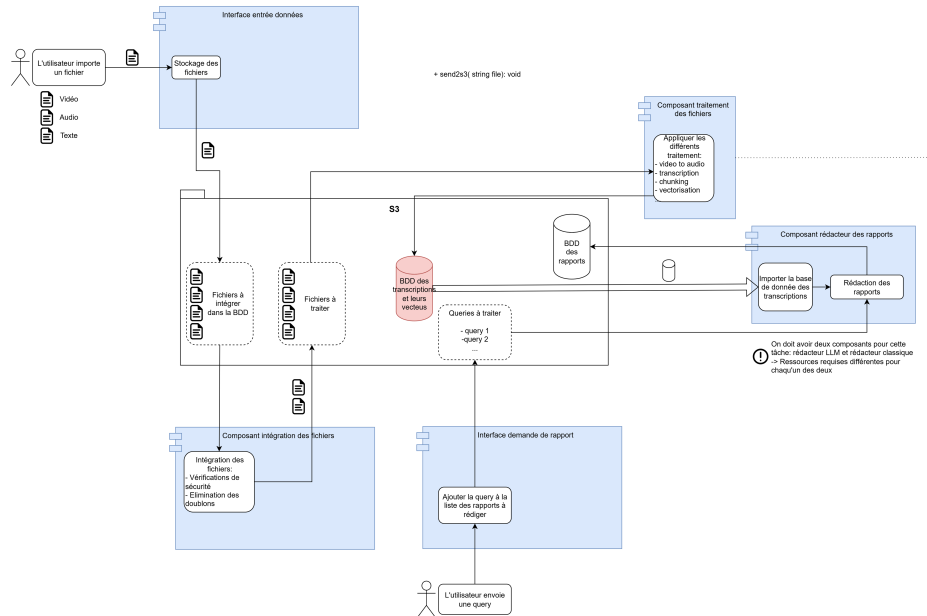


Fig. 2.3 Diagramme d'architecture de conception du projet Tech Event Expert

6 Travail en équipe

Après la conception du projet et sa validation par notre tuteur, nous nous sommes intéressés à l'élaboration du plan de travail et à la répartition des tâches. L'objectif restait de partager équitablement la charge de travail entre Idriss et moi, tout en limitant les dépendances mutuelles.

Pour ma part, j'ai pris en charge le développement du processus de traitement des fichiers multimédias, depuis leur entrée jusqu'à la construction de la base de données textuelle et vectorielle. Idriss, de son côté, a développé la suite du système exploitant cette base pour offrir à l'utilisateur une interface générant des rapports et la possibilité d'interagir sous forme de dialogue avec un LLM.

Nous avons ensuite rédigé la liste des sous-tâches nécessaires à la réalisation du projet. Chacune d'elles a été représentée sous forme de carte dans une méthode Kanban, mise en place via le site web Trello, ce qui nous a permis d'assurer une meilleure synchronisation, suivre et visualiser clairement l'état d'avancement du projet.

Pour la gestion du code, nous avons utilisé `GitLab`, ce qui nous a permis de fusionner facilement nos contributions respectives.

Enfin, nous avons organisé la période du stage en trois phases principales :

1. **Phase de développement et de recherche** (3 mois) : développement des fonctionnalités prévues, avec les expérimentations nécessaires.
2. **Phase d'aménagement et de documentation** (1 mois) : arrêt complet du développement pour se concentrer sur la structuration du code et la rédaction de la documentation.
3. **Phase finale** (1 mois) : préparation du présent rapport et de la soutenance, ainsi que corrections mineures sur les étapes précédentes.

7 Attendus techniques du stage

Comme on peut le déduire des sections précédentes, après la phase de conception du projet, mon sujet de stage s'est précisé et les tâches à réaliser se sont nettement clarifiées.

Mon rôle consiste à développer un système capable de prendre en entrée des fichiers audio et vidéo, et d'en construire une base de données robuste, exploitable par un système de RAG ou directement par un LLM.

Un autre aspect essentiel de ma mission est d'optimiser l'informativité des données. En effet, une partie significative des informations est perdue ou dégradée lors du passage des vidéos et fichiers audio vers les modèles LLM. L'objectif est donc de mettre en place des techniques permettant de préserver, voire de récupérer, un maximum de ces informations perdues.

En parallèle du développement technique, je serai également amené à documenter l'ensemble de mon travail afin de le rendre compréhensible et réutilisable par d'autres personnes, qu'il s'agisse de collègues, de futurs développeurs ou d'équipes externes. Cette documentation couvrira à la fois l'architecture générale du système, le fonctionnement détaillé des composants, ainsi que les procédures d'installation et d'utilisation.

Pour garantir la pérennité et la maintenabilité du projet, il est essentiel de respecter les normes de programmation en vigueur, de veiller à la clarté du code et d'adopter une structure

modulaire et cohérente. Ces bonnes pratiques faciliteront la compréhension, la modification et l'évolution du système à long terme, tout en réduisant les risques d'erreurs et les coûts de maintenance.

Travaux réalisés

1 Recherche bibliographique

Au début et tout au long du projet, une recherche bibliographique approfondie a été menée afin d'identifier les outils et techniques les plus adaptés à déployer. Cette recherche a couvert l'ensemble du périmètre du projet, allant des méthodes de traitement des données jusqu'aux différents systèmes de RAG. Elle a été largement facilitée par l'utilisation d'outils d'intelligence artificielle générative, tels que Consensus[11], qui ont permis d'accélérer l'exploration et la synthèse des sources pertinentes.

Dans cette section, nous nous concentrons exclusivement sur ma partie du projet, relative au traitement des données en amont de la construction de la base de données. L'objectif est de présenter directement les conclusions issues de cette analyse, sans entrer dans un examen exhaustif de la littérature ni dans une discussion détaillée des chiffres comparatifs. Pour les lecteurs intéressés par des éléments de comparaison ou par des références supplémentaires ayant guidé nos choix méthodologiques, des ressources bibliographiques complémentaires sont proposées en annexes A.

1.1 Technologies de reconnaissance vocale automatique (ASR)

1.1.1 Problématique

Nous avons été amenés à recourir à la reconnaissance vocale automatique, c'est-à-dire au processus de transcription de la parole en texte. Ce domaine, particulièrement vaste, a suscité l'attention de la communauté scientifique depuis la seconde moitié du XXe siècle et demeure aujourd'hui en constante évolution.

De manière générale, les approches développées pour cette tâche peuvent être regroupées en deux grandes familles : les modèles statistiques et les modèles basés sur les réseaux de neurones. Les méthodes statistiques, historiquement dominantes, présentent l'avantage d'être rapides et peu coûteuses en ressources, mais elles se révèlent souvent limitées en termes de précision, notamment face à la variabilité et à la complexité du langage parlé. À l'inverse, les approches neuronales, plus récentes, offrent des performances nettement supérieures en termes de justesse et de robustesse, mais au prix d'une plus grande complexité et de besoins computationnels plus importants.

Dans ce contexte, la mise en place d'un système de reconnaissance vocale requiert deux étapes décisionnelles essentielles. Il s'agit d'abord de choisir l'approche générale à privilégier — statistique ou neuronale — en fonction des contraintes du projet, qu'elles soient liées à

la rapidité d'exécution, à la précision attendue ou aux ressources disponibles. Une fois cette orientation définie, il convient de sélectionner le modèle spécifique le plus adapté à la tâche, chaque famille proposant une variété d'architectures et d'algorithmes aux caractéristiques distinctes.

1.1.2 Choix technique

Dans le cadre de notre projet, la contrainte principale ne réside pas dans la vitesse de traitement. En effet, nous ne développons pas un système de communication *temps réel* où un utilisateur attend une réponse immédiate, comme c'est le cas pour un assistant vocal ou un système de traduction simultanée. Ainsi, un temps de calcul plus long n'affecte pas l'expérience utilisateur.

En revanche, la précision des prédictions est un critère essentiel pour atteindre nos objectifs. Les erreurs de transcription pourraient compromettre la qualité des données en aval, ou fausser l'interprétation finale. Pour cette raison, nous orientons notre travail exclusivement vers les méthodes basées sur l'apprentissage profond (*deep learning*), qui ont démontré des performances supérieures en reconnaissance vocale sur de larges ensembles de données, notamment avec des architectures *end-to-end*, bien que plus coûteuses en calcul, offrent un taux d'erreur significativement réduit et une meilleure généralisation, ce qui correspond à nos exigences.

Quant au modèle de transcription. Le choix final pour notre déploiement s'est porté sur *Whisper*. Plusieurs raisons motivent cette décision.

Tout d'abord, *Whisper* est un modèle entièrement open source. Cela signifie qu'en l'utilisant, nous ne dépendons d'aucun service propriétaire externe, ce qui renforce notre autonomie technologique et garantit la maîtrise complète des données traitées. Cette indépendance est un atout majeur dans une perspective de souveraineté numérique et de pérennité de la solution.

Par ailleurs, *Whisper* bénéficie déjà d'une intégration dans le service *Groq*, qui permet de l'exécuter de manière optimisée sans que nous ayons à assurer nous-mêmes son déploiement ou sa maintenance sur une infrastructure dédiée. Cette disponibilité en tant que service performant réduit considérablement la complexité opérationnelle et le temps nécessaire à la mise en production.

Enfin, notre projet cible principalement les conférences scientifiques, lesquelles se tiennent très fréquemment en anglais. Sur cette langue, *Whisper* présente des performances élevées et une robustesse appréciable en conditions réelles. De plus, bien qu'optimisé pour l'anglais, il conserve des performances compétitives sur d'autres langues, ce qui assure une flexibilité importante en cas d'événements multilingues.

Avantages de Whisper pour les conférences scientifiques

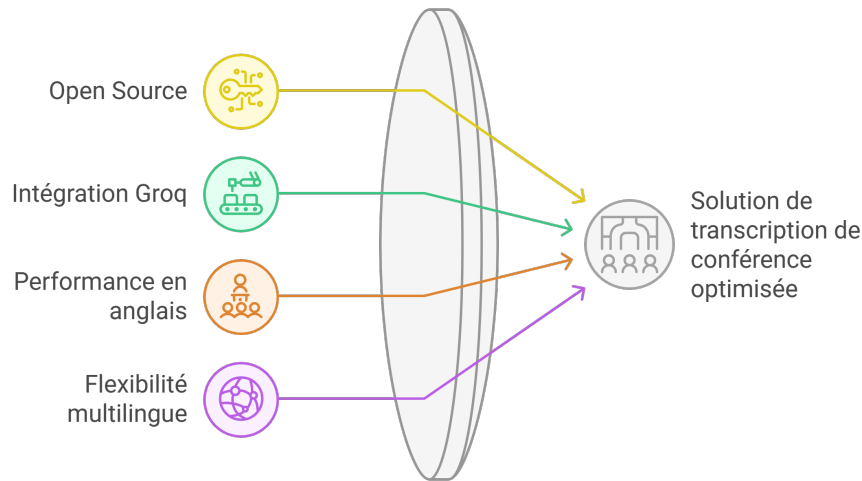


Fig. 3.1 Graphique expliquant les raisons poussant au choix d'utilisation de whisper pour la transcription dans le projet Tech Event Expert

1.2 Segmentation du texte

1.2.1 Problématique

Dans le cadre de la préparation des données destinées à être exploitées par un système de RAG, une étape essentielle consiste à segmenter ou découper les documents en unités plus petites et exploitables. Cette segmentation vise à structurer l'information de manière à la rendre plus facilement indexable et interrogeable, tout en préservant autant que possible la cohérence du contenu.

Ce découpage peut être réalisé selon trois approches principales. La première, stricte, repose sur l'utilisation d'une fenêtre glissante, avec ou sans chevauchement, permettant de produire des segments de taille fixe. Une alternative simple pour les documents bien structurés consiste à découper directement selon la structure naturelle du texte, par exemple au niveau des paragraphes ou des titres. Cette méthode a l'avantage d'être rapide et facile à mettre en œuvre, mais elle présente une limite importante : elle ne tient pas compte du contexte global et risque ainsi de fragiliser la pertinence des segments obtenus.

Pour pallier cette faiblesse, deux autres familles de méthodes cherchent à intégrer davantage de cohérence contextuelle. D'un côté, les approches statistiques, qui s'appuient sur des mesures telles que la fréquence ou la distribution des mots, permettent d'identifier des points de rupture pertinents dans le texte. De l'autre, les méthodes fondées sur les réseaux de neurones capturent les relations sémantiques de manière plus fine, offrant une segmentation capable de préserver un contexte plus complexe et plus riche.

1.2.2 Choix technique

Dans ce travail, l'objectif principal est d'assurer la meilleure qualité possible des données de sortie, au-delà de la vitesse de traitement. Cette priorité nous a naturellement orientés vers l'adoption de méthodes de deep learning, qui permettent de capturer des relations complexes et de produire des résultats plus précis, contrairement aux méthodes purement algorithmiques, heuristiques ou statistiques, qui peuvent être plus rapides mais offrent souvent des performances limitées sur des tâches complexes. La méthode que nous avons choisie pour la segmentation repose sur l'utilisation de modèles de langage de grande taille (LLM).

Il convient de noter que notre expérimentation de cette approche a débuté avant sa parution dans un article scientifique, ce qui montre que cette démarche s'inscrit dans une réflexion méthodique et indépendante. Plus spécifiquement, notre approche combine l'exploitation des capacités d'un LLM avec des opérations statistiques ciblées, afin d'améliorer la précision et la robustesse de la segmentation.

Cette méthode, qui allie apprentissage profond et traitement statistique, sera présentée en détail dans la section suivante 2.5, où nous exposerons l'ensemble des étapes et des considérations techniques qui sous-tendent son fonctionnement.

1.3 Diarisation

La diarisation est une tâche fondamentale en traitement automatique de la parole, consistant à segmenter un enregistrement audio en fonction des locuteurs. L'objectif est de répondre à la question « qui parle quand ? ». Elle est particulièrement utile dans des contextes où plusieurs intervenants prennent la parole. Cette étape permet de structurer les données audio avant d'appliquer d'autres traitements, comme la reconnaissance automatique de la parole (ASR) ou l'analyse conversationnelle.

Parmi les outils les plus utilisés pour la diarisation, `pyannote.audio` occupe une place importante. `pyannote` est une bibliothèque open source développée en Python, dédiée à l'analyse de la parole. Sa conception modulaire repose sur PyTorch, une framework open source de deep learning très répandue dans le domaine de la recherche et maintenue par une importante communauté de chercheurs et de développeurs.

Le fonctionnement de `pyannote` repose sur une chaîne de traitements intégrant plusieurs modules. Dans un premier temps, le signal audio est découpé en segments courts qui sont transformés en représentations spectrales. Ces représentations sont ensuite encodées par des modèles neuronaux spécialisés dans l'extraction d'embeddings de locuteurs, c'est-à-dire des vecteurs caractérisant la voix indépendamment du contenu linguistique. Ces embeddings sont ensuite regroupés par des algorithmes de clustering afin d'assigner chaque segment à un locuteur. `Pyannote` intègre également des étapes de raffinement comme la détection de changements de locuteurs ou l'ajustement temporel des segments.

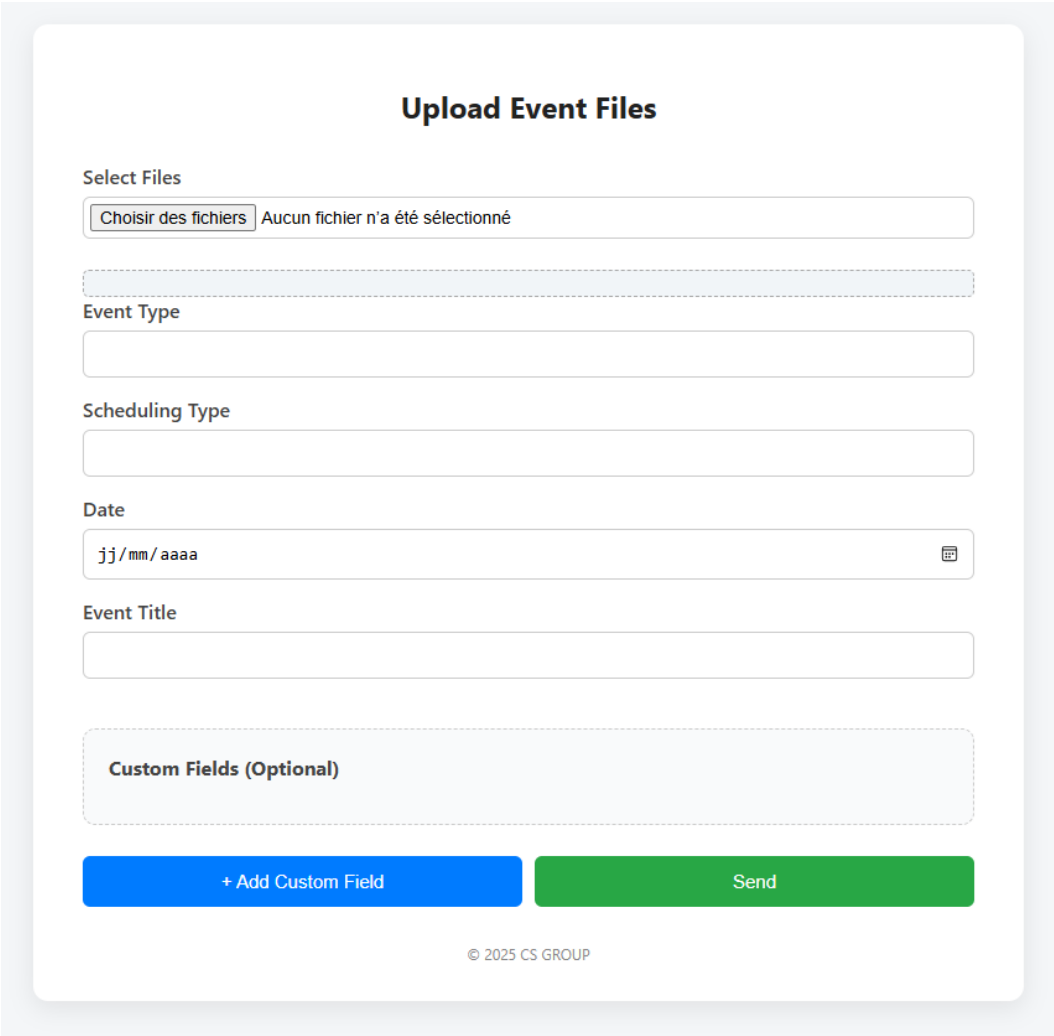
Nous privilégions l'utilisation de `pyannote` principalement pour deux raisons : d'une part sa simplicité d'utilisation, grâce à une API claire qui permet de mettre en place rapidement un système de diarisation sans expertise approfondie en apprentissage profond. D'autre part ses bonnes performances, largement démontrées dans les benchmarks internationaux tels que DIHARD ou AMI Meeting Corpus. En combinant accessibilité, flexibilité et robustesse,

pyannote s'impose ainsi comme une solution de référence dans nos travaux sur la diarisation.

2 Construction du pipeline

2.1 Processus d'intégration des nouveaux fichiers

Le processus décrit a pour objectif la préparation et l'intégration de fichiers multimédias dans une base de données. Initialement conçu sous la forme d'un script nécessitant une modification manuelle pour spécifier les fichiers à intégrer, il a été par la suite enrichi par la mise en place d'une interface web minimaliste (Fig 3.2) reposant sur la bibliothèque Flask, une bibliothèque simpliste dans Python qui permet de créer des interfaces graphiques en peu de temps et avec peu de complexité.



The screenshot shows a web form titled "Upload Event Files". It contains several input fields: "Select Files" with a button "Choisir des fichiers" and the text "Aucun fichier n'a été sélectionné"; "Event Type"; "Scheduling Type"; "Date" with a placeholder "jj/mm/aaaa" and a calendar icon; "Event Title"; and a dashed box for "Custom Fields (Optional)". At the bottom, there are two buttons: "+ Add Custom Field" (blue) and "Send" (green). A copyright notice "© 2025 CS GROUP" is visible at the very bottom.

Fig. 3.2 Un aperçu de l'interface web permettant d'ajouter des nouveaux fichiers média à la base de données

L'intégration repose sur une contrainte stricte relative aux fichiers d'entrée. Parmi les conditions exigées figure l'existence d'une liste d'extensions acceptées, limitée aux seuls for-

mats audio et vidéo. Tout fichier dont l'extension ne correspond pas à cette liste est automatiquement rejeté. En complément, chaque fichier média doit obligatoirement être accompagné d'un fichier de métadonnées au format `JSON`. Ce dernier porte de manière impérative le nom exact `meta-data.json` et contient des informations descriptives relatives à l'enregistrement, telles que le nom de la conférence, sa date de tenue ou d'autres attributs nécessaires à l'identification et à la classification du média. La présence conjointe de ces deux fichiers, et uniquement de ces deux fichiers, est requise. Ainsi, toute divergence, qu'il s'agisse d'un nombre de fichiers différent de deux ou d'une erreur dans l'appellation du fichier de métadonnées, entraîne le rejet du dossier concerné.

Une fois les conditions de validation satisfaites, l'utilisateur doit spécifier le chemin vers le répertoire contenant le couple constitué du fichier média et du fichier `meta-data.json`. Le système procède alors au renommage du dossier afin de lui attribuer une désignation unique, garantissant ainsi l'absence de collisions de noms lors du stockage. Finalement, le répertoire ainsi préparé est transféré vers le serveur S3, dans le répertoire `/upload`.

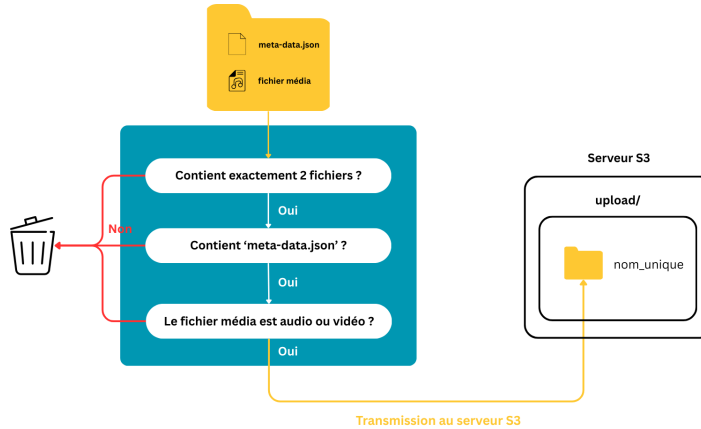


Fig. 3.3 Rerépresentation graphique du fonctionnement du processus d'intégration des nouveaux fichiers dans la base de données

2.2 Processus de pré-traitement

Ce module constitue une étape préliminaire essentielle dans le pipeline de traitement des fichiers. Son rôle principal est double : d'une part, il assure l'unicité des fichiers intégrés dans la base de données, et d'autre part, il procède à la normalisation de leur format afin de faciliter les traitements ultérieurs. Le processus débute par le téléchargement du répertoire `upload/`, qui contient les nouveaux fichiers issus du mécanisme d'intégration.

2.2.1 Algorithme d'unicité des fichiers

L'unicité des fichiers est garantie grâce à l'utilisation d'empreintes numériques générées par des fonctions de hachage. Le hachage (hashing) consiste à transformer un contenu en une chaîne de caractères de longueur fixe, de telle manière qu'une modification, même minime, du contenu initial entraîne une empreinte totalement différente [37]. De plus, la probabilité que deux fichiers distincts produisent le même hash est statistiquement négligeable. Cette propriété en fait une méthode rapide, peu coûteuse et particulièrement adaptée à la comparaison de fichiers [48].

Concrètement, chaque fichier média importé est d'abord haché, puis l'empreinte obtenue est sauvegardée dans un fichier annexe nommé `hash.txt`, placé dans le même répertoire que le fichier original. Le hash est ensuite comparé à deux ensembles de références : d'une part, aux empreintes des fichiers présents dans le même lot récemment importé depuis le serveur S3, et d'autre part, aux empreintes déjà stockées dans le répertoire `ready/`, lequel contient les fichiers validés et intégrés à la base de données. Cette double comparaison permet à la fois de détecter des duplications internes au lot courant et d'éviter la réintégration de fichiers déjà traités.

En cas de duplication avérée, le fichier concerné est immédiatement supprimé afin de préserver l'intégrité et la cohérence des données. À l'inverse, si aucun doublon n'est détecté, le fichier est conservé pour la suite du pipeline, accompagné de son fichier `hash.txt`. En fin de traitement, l'empreinte est transférée dans le répertoire `ready/` sur le serveur S3, garantissant la traçabilité et la pérennité du mécanisme de déduplication.

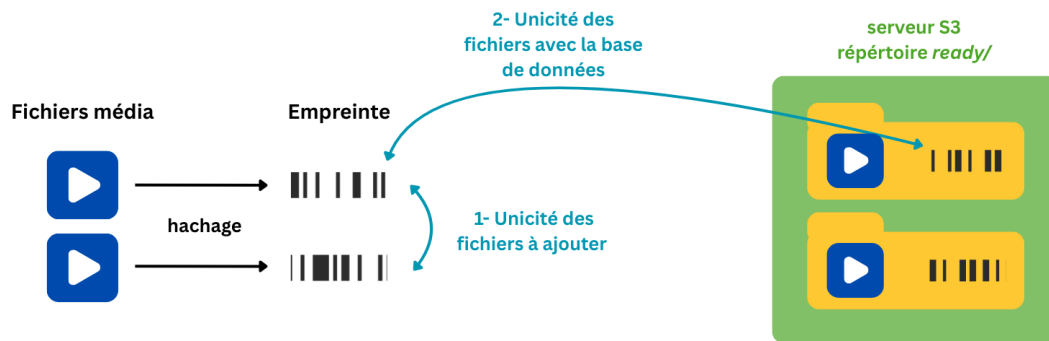


Fig. 3.4 Graphique illustrant l'algorithme de garantie d'unicité des fichiers dans la base de données par la méthode du hachage

2.2.2 Normalisation du format des fichiers

Afin d'améliorer l'expérience utilisateur, il est préférable d'accepter un large éventail de formats audio et vidéo. En effet, le format du fichier dépend souvent de l'appareil utilisé pour l'enregistrement, du support de stockage ou encore des méthodes de transfert appliquées. Imposer à l'utilisateur une conversion manuelle de ses fichiers avant de les soumettre à notre système serait contraignant et peu ergonomique.

Dans le même temps, pour garantir un traitement standardisé, il est essentiel de disposer d'un format unique que l'on puisse traiter de manière homogène. Cette uniformisation réduit considérablement le risque d'erreurs et rend plus transparents les traitements appliqués à chaque étape.

Pour concilier ces deux besoins, nous avons choisi d'accepter un large éventail de formats en entrée et d'effectuer la standardisation côté serveur. Tous les fichiers sont ainsi convertis en audio au format `.wav` grâce à l'outil open source `FFmpeg`. Ce choix n'est pas anodin : le format WAV est non compressé, ce qui garantit l'intégrité et la richesse informationnelle du fichier d'origine, tout en offrant une lecture rapide et fiable.

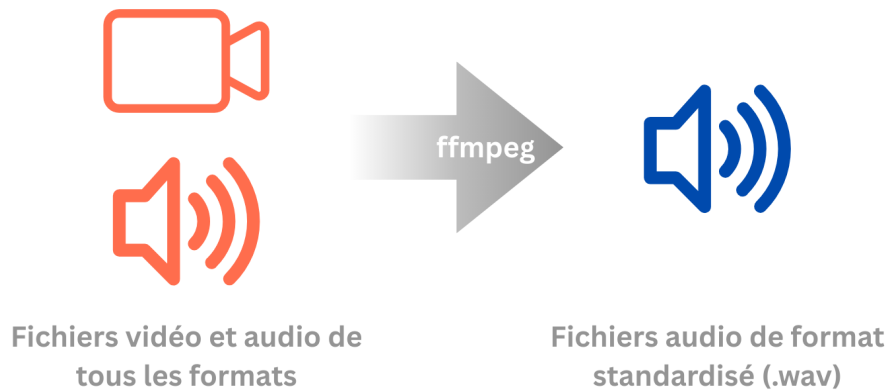


Fig. 3.5 Graphique expliquant la normalisation de format des fichiers multimédia avant tout traitement

2.2.3 Post pré-traitement

Après la vérification de l'unicité du fichier à ajouter et la normalisation de son format, on obtient un dossier standardisé contenant quatre éléments :

- le fichier original, conservé avec son format et son nom d'origine ;
- le fichier `meta-data.json` ;
- le fichier `hash.txt`, qui contient l'empreinte du fichier original ;
- le fichier `media.wav`, représentant la version normalisée du média.

Ensuite, ce répertoire est transféré vers le serveur S3 sous un nom généré et unique, à l'intérieur du répertoire `pre-processed/`, afin d'être utilisé par les étapes de traitement en aval.

Les étapes de traitement suivantes s'attendent donc toujours à retrouver le fichier média d'entrée sous le nom fixe `media.wav`.

Le processus en cours nettoie également le répertoire `upload/` du serveur S3, supprimant tout son contenu, que les fichiers aient été traités et transférés vers `pre-processed/` ou éliminés en raison d'une incompatibilité.

2.3 Processus de transcription automatique

Au début du processus, la liste des fichiers à traiter est récupérée depuis le serveur S3, dans le répertoire `pre-processed/`. Une fois téléchargés, les fichiers sont traités successivement sur la même machine par trois processus : transcription, diarisation et découpage. Il est important de noter qu'aucun échange intermédiaire avec le serveur S3 n'a lieu entre ces étapes.

Pour la transcription, le modèle Whisper est utilisé. Ce modèle prend en entrée un fichier audio et produit en sortie des segments de transcription de durée variable, chacun accompagné de ses délimiteurs temporels. Par exemple, un fichier audio peut générer les segments suivants :

[00:00:00 - 00:00:07] Bonjour, comment trouvez vous ce rapport ?

[00:00:07 - 00:00:15] Ce rapport fait preuve d'une rédaction rigoureuse.
[00:00:15 - 00:00:23] Je vous remercie pour votre remarque.

Plutôt que d'exécuter le modèle localement, nous privilégions son utilisation via le service serverless Groq, ce qui permet de bénéficier de performances optimisées. Cependant, cette approche impose une limite sur la taille des fichiers pouvant être envoyés pour traitement.

Pour contourner cette contrainte, les fichiers audio sont découpés en plusieurs segments plus petits, chaque segment étant transcrit séparément avant d'être recombéné. La principale difficulté réside dans le fonctionnement du modèle Whisper, qui utilise une « fenêtre glissante » de 30 secondes avec chevauchement. Un découpage strict des fichiers entraînerait la perte d'informations aux extrémités des segments, ce qui pourrait réduire la précision des transcriptions.

Afin de résoudre ce problème, nous avons proposé une méthode de découpage garantissant un chevauchement exact de 30 secondes entre les segments. Lors de la reconstruction du fichier complet, les prédictions correspondant aux dernières 15 secondes du segment précédent et aux premières 15 secondes du segment suivant sont supprimées. Cette stratégie permet de simuler le comportement d'une fenêtre glissante sur l'ensemble du fichier audio, assurant ainsi une couverture complète et une précision de transcription équivalente à celle obtenue sur un fichier non découpé.

À la sortie de ce processus, un fichier `transcription.json` est généré. Ce fichier contient l'ensemble des segments de transcription et est enregistré dans le même répertoire que le fichier audio d'origine.

2.4 Processus de diarisation

Pour rendre la transcription plus informative et exploitable, nous procédons à une attribution des segments à chaque intervenant, ce que l'on appelle l'alignement avec la diarisation. Concrètement, en parallèle de la transcription, nous appliquons un modèle de diarisation `pyannote`[\[8\]](#) sur le fichier audio. Ce modèle extrait des caractéristiques sonores propres à chaque locuteur et regroupe les segments qui semblent provenir de la même personne. À l'issue de cette étape, nous obtenons des segments temporels pour chaque interlocuteur, indiquant quand chacun parle.

Par exemple, si nous reprenons l'exemple de discussion entre Alice et Bob. La transcription de Whisper pourrait produire les segments suivants :

[00:00:00 - 00:00:07] Bonjour, comment trouvez vous ce rapport ?
[00:00:07 - 00:00:15] Ce rapport fait preuve d'une rédaction rigoureuse.
[00:00:15 - 00:00:23] Je vous remercie pour votre remarque.

À ce stade, nous ne savons pas qui dit quoi. Après la diarisation, nous obtenons :

Segment 1 [00:00:00 - 00:00:07] Alice

Segment 2 [00:00:07 - 00:00:15] Bob
 Segment 3 [00:00:15 - 00:00:23] Alice

La dernière étape consiste à aligner la transcription avec les résultats de la diarisation, pour produire un texte final structuré sous forme de dialogue :

Alice [00:00:00 - 00:00:07] Bonjour, comment trouvez vous ce rapport ?
 Bob [00:00:07 - 00:00:15] Ce rapport fait preuve d'une rédaction rigoureuse.
 Alice [00:00:15 - 00:00:23] Je vous remercie pour votre remarque.

Grâce à ce processus, nous passons d'une simple transcription linéaire à une conversation structurée, où chaque segment de texte est associé à son interlocuteur, ce qui rend le contenu plus clair et exploitable.

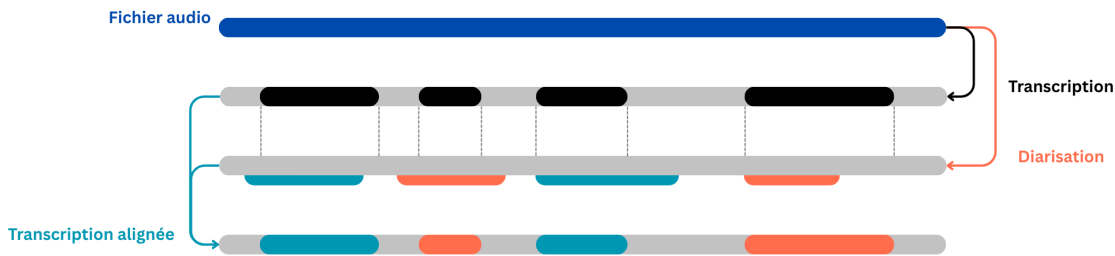


Fig. 3.6 Graphique expliquant le processus de transcription et de diarisation des fichiers audio employé dans le projet Tech Event Expert

Il est à noter, néanmoins, que la limite principale de cette méthode réside dans le fait que nous n'obtenons pas d'informations explicites sur les interlocuteurs, comme leurs prénoms ou identités réelles. Par exemple, dans une conversation entre Alice et Bob, la diarisation permet seulement de distinguer deux voix différentes et de les attribuer de manière cohérente aux segments correspondants, mais elle ne permet pas de savoir qu'une voix correspond précisément à Alice ou à Bob. Les segments sont donc identifiés de manière générique (Locuteur 1, Locuteur 2, etc.), et toute association avec des identités réelles nécessiterait un traitement ou un marquage supplémentaire.

Remarque :

Les segments produits par la diarisation et ceux issus de la transcription ne correspondent pas exactement en termes de délimitation temporelle, comme illustré dans le graphique 3.6. Dans notre approche, nous nous basons principalement sur les segments de transcription, car Whisper est un modèle plus grand, entraîné sur davantage de données que pyannote, et fournit donc des résultats temporels plus précis. Pour chaque segment de transcription, nous assignons le label du locuteur correspondant au segment de diarisation qui recouvre la plus grande partie de ce segment. Si un segment de transcription n'intersecte aucun segment prédit par Pyannote, nous labélisons l'interlocuteur comme inconnu, avec le label "unknown".

2.5 Processus de découpage

À l'issue de l'étape précédente, nous disposons d'un texte structuré sous forme de conversation. Pour pouvoir l'exploiter efficacement dans un système de recherche, nous procédons à un découpage sémantique en chunks (morceaux de texte), en veillant à conserver la cohérence sémantique de chaque segment.

Pour répondre à cette problématique, nous avons conçu une méthode innovante qui délègue la tâche de segmentation à un modèle de langage (LLM). L'algorithme mis en œuvre repose sur deux étapes principales.

Dans un premier temps, le texte est découpé de manière stricte à l'aide d'une fenêtre glissante de taille fixe avec chevauchement, tout en conservant les délimiteurs temporels de chaque segment. La taille de cette fenêtre, d'environ dix minutes, a été choisie de manière à ce qu'elle soit suffisamment large pour capturer le contexte et le sujet traité au sein de la transcription.

Ensuite, pour chaque partie découpée, le LLM est sollicité afin d'identifier un ou plusieurs points où le sens du texte semble changer. Bien qu'il soit rarement possible - même pour un humain - de déterminer avec certitude l'instant exact d'un changement thématique dans une conversation, le modèle peut proposer une localisation approximative, à laquelle nous associons une marge d'incertitude.

Afin de formaliser cette marge, nous transformons les prédictions du LLM en données probabilistes. Pour chaque fenêtre, le modèle effectue plusieurs itérations de prédiction, ce qui génère une distribution fréquentielle indiquant combien de fois chaque position a été considérée comme un point de séparation potentiel.

Ces données sont ensuite lissées par interpolation en utilisant la méthode KDA (Kernel Density Approximation), une méthode qui permet de générer une courbe d'interpolation à partir d'un nuage de points[44]. Les maxima locaux de la courbe obtenue correspondent alors aux positions les plus probables de changement de sens dans le texte, et sont retenus comme frontières de segmentation. ce qui nous permet de faire le découpage et d'avoir les chunks finaux, comme illustré dans la figure 3.7.

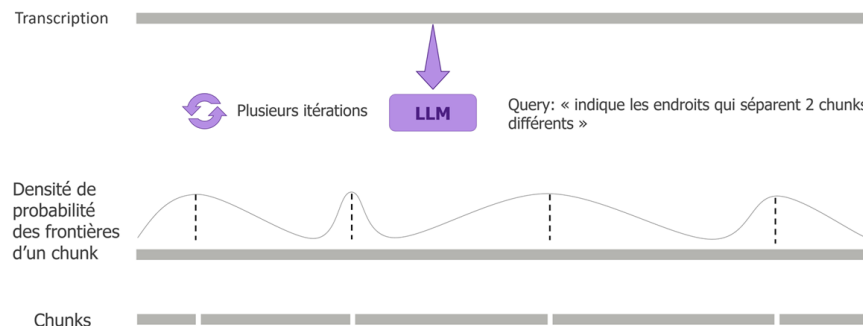


Fig. 3.7 Graphique explicatif de l'algorithme de segmentation utilisé lors du projet Tech Event Expert

Chaque chunk est ensuite vectorisé à l'aide du modèle open source `inf-retriever-v1-1.5b`,

qui se classe parmi les meilleurs modèles de vectorisation dans les benchmarks dédiés aux systèmes de RAG.

Nous stockons alors le texte des chunks dans une base NoSQL MongoDB, les vecteurs correspondants dans la base vectorielle Milvus et nous assurons le lien entre les deux via un identifiant commun.

Ainsi, lors d'une recherche, la requête est vectorisée puis envoyée à Milvus, qui retrouve rapidement les vecteurs les plus proches. Ensuite, grâce à l'identifiant associé, nous récupérons le texte correspondant dans MongoDB.

Par exemple, si un utilisateur recherche « résumé du rapport financier », Milvus identifie les chunks les plus proches sémantiquement (ex. un chunk vectorisé contenant « bilan économique trimestriel »), puis MongoDB fournit immédiatement le texte original lié à ce vecteur.

Cette architecture combine le meilleur des deux mondes :

- Milvus assure une recherche vectorielle rapide et efficace,
- MongoDB gère le stockage fiable et optimisé du texte de taille très variable.

En effet, Milvus n'est pas conçu pour stocker du texte brut et n'est pas optimisé pour gérer à la fois vecteurs et contenu textuel, d'où ce découplage.

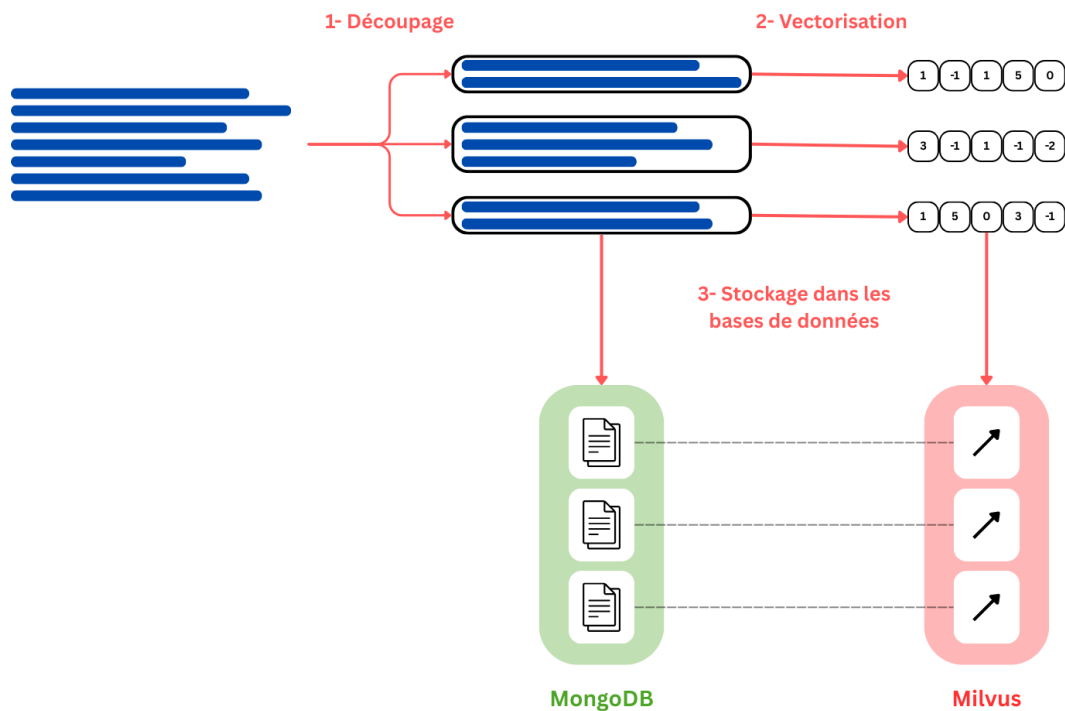


Fig. 3.8 Graphique expliquant l'algorithme de découpage textuel et de construction de la base de données du projet Tech Event Expert

De manière analogue à l'étape finale du processus précédent, le répertoire **pre-processed/** est supprimé. Les fichiers traités sont ensuite transférés dans le répertoire **ready/**, qui regroupe à la fois les éléments préexistants (tels que le hash et les métadonnées) ainsi que les nouveaux fichiers générés au cours du présent processus, notamment la transcription, la diarisation, la transcription alignée et les segments (chunks).

Remarque :

Chaque segment de texte produit à la suite du découpage présenté dans cette partie reste associé au fichier audio ou vidéo original. Cette association est rendue possible grâce aux métadonnées que nous construisons lors de l'ajout d'un élément à la base de données, ce qui permet de retrouver à tout moment le fichier source correspondant.

2.6 Post-traitement

Dès la fin du traitement des fichiers et leur intégration dans les bases de données, il devient possible d'effectuer une recherche d'information. Le système - rappelons-le - repose sur trois sources de données complémentaires : Milvus, qui stocke les vecteurs représentant les segments textuels (chunks) ; MongoDB, qui conserve les segments textuels eux-mêmes ainsi que leurs métadonnées ; et un serveur S3, qui contient l'ensemble des fichiers, notamment le fichier média original sur lequel reposent les traitements.

L'algorithme de recherche proposé suit un principe analogue à celui des moteurs de recherche classiques. Une requête textuelle, qu'il s'agisse d'une question, d'une thématique ou d'un ensemble de mots-clés, est tout d'abord vectorisée à l'aide du même modèle que celui utilisé lors de la construction des vecteurs dans Milvus (**inf-retriever-v1-1.5b**). Le vecteur obtenu est ensuite comparé aux vecteurs stockés dans Milvus afin d'établir un classement de similarité. Les résultats les mieux classés sont alors sélectionnés pour une analyse plus approfondie.

Pour chaque vecteur retenu, une recherche est effectuée dans MongoDB à l'aide de sa clé secondaire, permettant de récupérer le segment textuel associé ainsi que ses métadonnées. Ces métadonnées fournissent notamment le chemin d'accès au fichier original stocké dans le serveur S3, ainsi que l'intervalle temporel correspondant au chunk dans le média. Afin d'illustrer et de valider cette étape, un moteur de recherche de démonstration a été développé (Fig 3.9). Celui-ci permet non seulement d'interroger le système, mais également de visualiser et de lire le fichier média correspondant côté client.

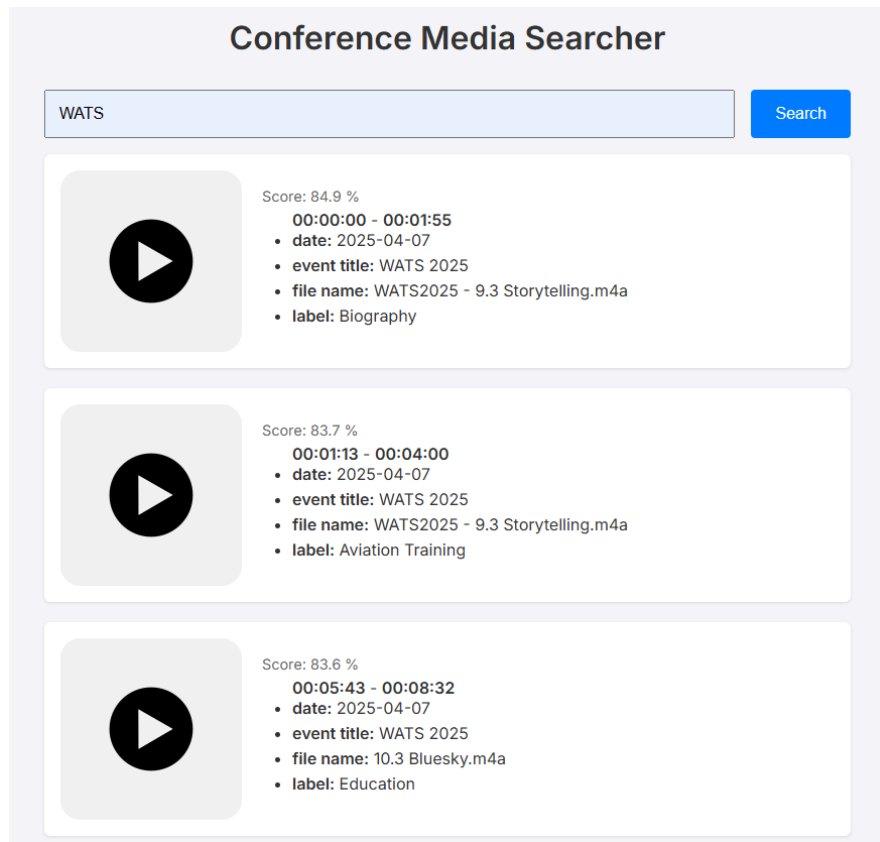


Fig. 3.9 Aperçu du moteur de recherche de démonstration utilisant la base de données développée dans le cadre du projet Tech Event Expert

3 Architecture de la base de données

3.1 Choix des bases de données

Notre choix de Milvus s'explique par les performances qu'elle offre dans le traitement et la recherche vectorielle. Elle s'est révélée particulièrement efficace lors de son intégration dans MyCsGroupAssistant, où elle a démontré sa fiabilité et sa rapidité. C'est cette expérience concluante qui a motivé son adoption dans notre projet actuel.

Pour MongoDB, c'est la flexibilité qui a guidé notre décision. Nous avions besoin d'une base de données capable de s'adapter facilement aux évolutions du projet, sans contrainte de schéma rigide. Le modèle NoSQL de MongoDB répond parfaitement à cette exigence. Son interface graphique Compass a également facilité sa manipulation, rendant son utilisation à la fois pratique et intuitive.

Enfin, l'adoption d'Amazon S3 découle principalement de deux raisons : sa rapidité en tant que service de stockage, surtout lorsqu'il est acouplé des instances EC2, ce qui le cas dans notre projet. Et le fait que l'infrastructure de l'entreprise repose déjà sur AWS. Ce choix nous a donc permis de garantir à la fois efficacité et cohérence avec l'existant.

3.2 Schéma de données pour les transcriptions

Pour le stockage et la gestion des segments de texte, nous avons défini un schéma spécifique dans MongoDB, adapté aux besoins de notre projet. Chaque document de la collection représente un *chunk*.

- **`__id`** : identifiant unique généré automatiquement pour chaque chunk.
- **`chunk_text`** : le contenu textuel du segment.
- **`media_folder_name`** : le nom du dossier dans Amazon S3 contenant l'enregistrement source du chunk.
- **`chunk_start`** : la position de début du segment dans l'enregistrement d'origine.
- **`chunk_end`** : la position de fin du segment.
- **`chunk_number`** : un entier indiquant la position du chunk lorsque le texte a été découpé en plusieurs segments, ce qui permet de reconstituer l'ordre du contenu.
- **`label`** : un titre généré automatiquement par un LLM, permettant de donner une idée rapide du contenu du chunk.

Concernant Milvus, le schéma est plus simple puisqu'il est dédié exclusivement au stockage et à la recherche de vecteurs. Chaque entrée contient uniquement deux éléments :

- **`__id`** : l'identifiant unique du vecteur, identique à son chunk correspondant stocké dans MongoDB.
- **`vector`** : le vecteur numérique représentant le contenu textuel du chunk.

3.3 Indexation et recherche

Pour effectuer une recherche d'information dans la base de données, nous suivons l'algorithme suivant :

1. Identifier les chunks les plus pertinents à l'aide de Milvus, grâce à la recherche par similarité sur leurs vecteurs.
2. Récupérer les documents correspondants à ces chunks à l'aide de l'identifiant unique et commun `__id`. Cela permet d'obtenir les principales informations autour du chunk, y compris le contenu textuel lui-même.
3. Si nécessaire, récupérer également les données originales soumises à l'entrée, c'est-à-dire le fichier média avant traitement ainsi que les métadonnées associées, en utilisant le champ `media_folder_name`.

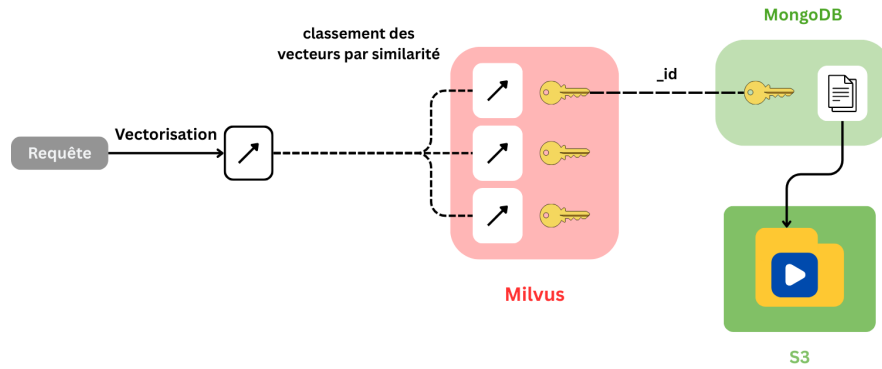


Fig. 3.1 Diagramme explicatif de l'algorithme envisagé pour la recherche d'un segment de texte et de son fichier média à partir d'une requête textuelle

4 R&T exploratoire

Dans cette section, nous présenterons certaines expériences qui avaient été initialement prévues dans le cadre du projet ou qui auraient été pertinentes à intégrer dans le processus de développement. Toutefois, en raison de contraintes temporelles ou de certaines limites techniques rencontrées, leur réalisation complète n'a pas pu aboutir. Ces expériences demeurent néanmoins importantes, car elles constituent des pistes d'amélioration et des perspectives de travail futur.

4.1 Vérification de sécurité du format du fichier

Lors de la phase de conception du processus de prétraitement, il avait initialement été prévu d'intégrer une fonctionnalité de sécurité supplémentaire. L'objectif de cette étape consistait à analyser la signature binaire du fichier afin de déterminer son véritable format, puis de comparer ce dernier avec l'extension déclarée. Une divergence entre les deux constituait un indicateur de corruption ou de tentative de soumission d'un fichier malveillant. Dans de tels cas, le fichier était automatiquement rejeté.

Cette approche avait déjà été appliquée avec succès en 2024 dans le cadre du projet *MyCSGroupAssistant*. Pour cette tâche, on a utilisé *Magika*[15], un modèle développé par Google, réputé pour sa rapidité, sa compacité et ses performances dans la classification de formats binaires. Les résultats obtenus avaient confirmé la pertinence de cette méthode, renforçant ainsi la robustesse du système face à des entrées potentiellement compromises.

Cependant, son application au projet *Tech Event Expert* s'est révélée inadaptée. En effet, la diversité des extensions traitées par *Magika* était très limitée, se restreignant principalement à des documents textuels tels que les fichiers `.txt` ou `.pdf`. Cette contrainte a conduit à une dégradation significative de la précision des prédictions : le modèle a produit des classifications erronées, parfois totalement fallacieuses. Devant cette inefficacité manifeste, la fonctionnalité a finalement été abandonnée pour ce projet.

4.2 Identification des interlocuteurs à l'aide de leur empreinte sonore

Dans le cadre ce travail, nous nous sommes intéressés à une utilisation complémentaire des fichiers médias issus des conférences : l'identification automatique des intervenants. Une telle fonctionnalité présente plusieurs intérêts. Elle permet par exemple de suivre l'évolution d'une entreprise en analysant les interventions récurrentes de l'un de ses représentants, ou encore de réduire le temps de recherche lorsqu'un même interlocuteur donne des présentations similaires dans différentes conférences.

Cependant, ce type d'approche soulève des enjeux liés au traitement de données personnelles, en particulier lorsqu'il s'agit d'empreintes vocales. Dans notre projet, les expérimentations menées restent strictement exploratoires et n'impliquent donc aucune exploitation dans un cadre opérationnel. Néanmoins, une éventuelle transposition industrielle de cette technologie devrait s'accompagner d'une attention particulière au respect des cadres légaux en vigueur.

La première expérience que nous avons conduite consiste à extraire une empreinte sonore propre à chaque interlocuteur, puis à comparer cette empreinte à celles obtenues dans d'autres fichiers médias afin de déterminer s'il s'agit ou non du même locuteur. Pour ce faire, nous avons utilisé le modèle pyannote, au sein d'un pipeline conçu pour éliminer les chevauchements de voix lorsque plusieurs intervenants parlent simultanément, ce qui améliore la précision de l'analyse. Chaque segment vocal correspondant à un locuteur est ensuite vectorisé avec un autre pipeline de vectorisation de pyannote, puis les vecteurs sont regroupés par clustering afin d'identifier les différents intervenants dans un même fichier.

Dans un second temps, nous avons procédé à des comparaisons inter-fichiers. Pour chaque groupe de vecteurs représentant un locuteur donné, nous avons calculé la moyenne de la similarité avec les vecteurs extraits d'autres fichiers. Les résultats montrent que la similarité demeure élevée lorsqu'il s'agit du même intervenant, tandis qu'elle reste faible pour des locuteurs différents. En fixant un seuil de similarité à 0.2, nous observons de manière générale que toutes les valeurs supérieures correspondent à une même identité vocale.

Cette approche s'est révélée être une réussite, offrant une précision satisfaisante dans la reconnaissance inter-fichiers des interlocuteurs. Les résultats obtenus confirment la robustesse de la méthode : le modèle parvient à distinguer correctement les identités vocales, même en présence de conditions acoustiques variées entre les enregistrements. Ces performances encourageantes démontrent que la mise en place d'un tel système est tout à fait réalisable.

4.3 Enrichissement d'identification des interlocuteurs (métadonnées exploitables dans les pages web descriptives des conférence)

La partie précédente 4.2 a permis de valider la faisabilité de la distinction entre différents interlocuteurs ainsi que leur identification à partir de leur empreinte sonore. Toutefois, cette identification demeure limitée : elle ne fournit qu'un identifiant technique, sans information complémentaire sur l'intervenant.

Nous nous sommes donc interrogés sur la possibilité d'enrichir notre base de connaissances en exploitant des informations personnelles supplémentaires relatives aux conférenciers. En pratique, les conférences sont très souvent accompagnées d'un site web présentant la liste

des intervenants, incluant g n ralement leur nom, pr nom, fonction et parfois leur entreprise. Nous avons ainsi cherch    concevoir une m thode pour extraire automatiquement ces donn es.

La premi re approche envisag e consistait   fournir le code HTML de chaque page du site web de la conf rence   un mod le de langage, afin qu'il en extraie directement les informations pertinentes. N anmoins, cette solution s'est rapidement heurt e   une contrainte technique : la taille importante des fichiers HTML emp chait leur traitement direct en entr e par le mod le.

Une seconde piste a  t  explor e   travers l'utilisation de mod les de vision, appliqu s cette fois non pas au code source mais   l'image des pages web. Cette m thode s'est r v l e efficace pour de petites zones visuelles (par exemple, l'extraction de noms sur des encarts r duits), mais elle a montr  de fortes limites d s lors qu'il s'agissait de traiter des pages compl tes. Dans ce cas, le mod le pouvait parfois identifier un pr nom, mais rencontrait de grandes difficult s   extraire correctement et de mani re coh rente l'ensemble des informations textuelles faisant des erreurs d'orthographe et confondant certaines lettres.

Finalement, nous avons retenu une troisi me solution, reposant de nouveau sur l'analyse du code HTML. Gr ce   la biblioth que `BeautifulSoup`, le code est transform  en texte brut, ce qui r duit consid rablement sa taille et permet ainsi de l'exploiter dans un LLM. Avec un param trage ad quat du prompt en entr e, cette approche s'est r v l e particuli rement efficace, offrant une extraction fiable et pr cise des donn es relatives aux conf renciers.

4.4 Identification des interlocuteurs dans un enregistrement

 tant donn  le succ s des exp riences men es dans les parties 4.3 et 4.2, nous avons entrepris de combiner les deux approches afin d'attribuer les informations personnelles des interlocuteurs (nom, pr nom, etc.)   leur empreinte sonore. Concr tement, cela permet d'associer directement un locuteur identifi  par son empreinte vocale aux donn es le concernant, chaque fois qu'il intervient au cours d'une conf rence. Cette int gration constitue un moyen de d passer la limite  voqu e dans la partie 2.4, o  les r sultats de diarisation restent anonymes et ne fournissent qu'une segmentation en « locuteur 1 », « locuteur 2 », etc.

Il est fr quent, au cours des conf rences, que les intervenants soient pr sent s en d tail par l'animateur ou par d'autres participants. L'id e explor e dans cette exp rience est donc de tirer parti de ces pr sentations. Plus pr cis ment, nous avons utilis  les transcriptions des enregistrements et demand    un mod le de langage de d tecter si une pr sentation de conf rencier y apparaissait, et, le cas  ch ant, d'associer cette pr sentation au bon locuteur (initialement identifi  uniquement par un code g n rique tel que « locuteur 1 »).

Toutefois, les r sultats de cette premi re approche se sont r v l s insatisfaisants. Lorsque seul le texte brut de la transcription est fourni, le mod le pr sente de forts biais : il g n re un nombre important d'hallucinations et  choue fr quemment   extraire correctement les informations ou   les attribuer au bon intervenant. M me avec un prompting strict et fortement orient , la qualit  de l'extraction reste limit e.

Afin d'am liorer cette m thode, nous avons propos  d'enrichir le prompt en y int grant les m tadonn es extraites pr alablement du site web de la conf rence. Cette assistance sup-

plémentaire devait fournir au modèle des points de référence fiables, en réduisant l’ambiguïté des transcriptions. Si cette seconde approche a effectivement permis une amélioration des résultats, l’attribution correcte des données demeure encore largement imparfaite : un taux élevé d’hallucinations persiste, compromettant la fiabilité de l’ensemble du processus.

Difficultés rencontrées et solutions

1 Prompt engineering

L'utilisation des modèles LLM (Large Language Model) et SLM (Small Language Model) a constitué un atout majeur dans ce projet. Néanmoins, leur paramétrage s'est révélé complexe : en l'absence d'un prompt soigneusement formulé et rigoureusement orienté, les modèles de langage présentent fréquemment des hallucinations ou des imprécisions significatives.

Une part importante du temps de développement a ainsi été consacrée à l'optimisation des prompts et des réglages des LLMs, un effort qui s'est parfois avéré plus chronophage que leur déploiement effectif.

Certaines stratégies, que nous n'avons pas pleinement exploitées durant le projet, peuvent néanmoins améliorer sensiblement leurs performances. Parmi elles, le découpage de la tâche en sous-tâches successives constitue une approche particulièrement prometteuse. Plutôt que de demander à un modèle unique d'assurer simultanément l'extraction des données, leur structuration et leur mise en forme au format JSON, il serait préférable de séquencer le processus en trois étapes distinctes, confiées à trois modèles spécialisés. L'un se concentrerait sur l'extraction, un second sur la structuration, et le dernier sur le formatage. Cette réduction de la complexité des tâches attribuées à chaque modèle permettrait d'améliorer la précision globale tout en diminuant le risque d'hallucinations.

2 Limitations du service Groq

Le serveur Groq s'est révélé particulièrement utile dans le cadre de ce projet, en facilitant l'accès à plusieurs modèles de deep learning, notamment Whisper pour la transcription et différents modèles de LLM. Le principal atout de ce service réside dans sa rapidité : il est spécifiquement optimisé pour des usages en temps réel, ce qui le rend très pratique, en particulier pour les tâches de transcription et de génération textuelle.

Cependant, cette orientation vers la vitesse constitue également sa principale limite. En se concentrant sur la performance en temps de calcul, Groq ne propose que des modèles de taille relativement réduite, n'excédant pas 90 milliards de paramètres. Cette contrainte limite donc l'accès à des modèles plus performants et plus robustes, qui pourraient apporter une meilleure précision et une plus grande consistance dans certains cas d'usage.

Pour surmonter cette problématique, une alternative serait de se tourner vers des concurrents offrant des services similaires, mais avec une plus large diversité de modèles et une

capacité à intégrer des architectures plus volumineuses. C'est le cas, par exemple, de Fireworks.ai, qui propose une vaste palette de modèles, incluant non seulement des LLM de différentes tailles, mais aussi des modèles de diffusion pour la génération d'images.

3 Limitations de centralisation de la base de données vectorielle

Après traitement, l'extraction d'informations repose sur l'utilisation conjointe de deux bases de données : Milvus et MongoDB. Cette dualité permet d'accélérer les recherches par rapport à une solution reposant uniquement sur MongoDB, qui se révèle relativement lente pour ce type d'opérations. Toutefois, cette architecture hybride présente également plusieurs limitations.

En effet, il serait idéal de pouvoir centraliser l'ensemble des données dans une base unique. Or, MongoDB souffre de performances limitées en recherche, tandis que Milvus, bien qu'optimisé pour la gestion et l'indexation de vecteurs, reste peu adapté au stockage de données textuelles, de fichiers JSON ou d'autres formats hétérogènes. Cette complémentarité forcée nous contraint donc à utiliser deux systèmes distincts.

Cette dualité introduit une problématique importante : la gestion de la cohérence entre les deux schémas de données. Il est indispensable de s'assurer manuellement qu'aucun enregistrement ne soit partiellement stocké dans l'une des bases sans que les informations correspondantes ne soient présentes dans l'autre, sous peine de créer des incohérences critiques dans le système.

Conclusions et perspectives

1 Bilan du travail réalisé

1.1 Objectifs initiaux et objectifs atteints

L'objectif initial du projet était double : valoriser les enregistrements réalisés par les participants aux conférences et offrir un moyen simple d'extraire des informations sur les thématiques abordées. Cet objectif s'est progressivement précisé avec la mise en place d'un système de RAG (Retrieval-Augmented Generation) fondé sur les transcriptions de ces enregistrements.

Concrètement, nous avons pu développer un système capable de transformer les enregistrements en transcriptions diarisées, ainsi qu'un chatbot exploitant ces transcriptions pour interagir avec l'utilisateur. Nous avons également conçu un service basé sur un LLM capable de générer des rapports techniques, de longueur variable, sur des sujets précis, en s'appuyant sur les discussions ayant eu lieu. Enfin, nous avons intégré un moteur de recherche thématique, permettant de retrouver rapidement des références en lien avec un domaine donné.

Les expérimentations menées sur ces outils ont démontré leur bon fonctionnement, mais aussi leur neutralité dans le transfert d'informations, ce qui représente une amélioration notable par rapport aux méthodes informelles de transmission orale (bouche à oreille).

Ainsi, on peut affirmer que les objectifs initiaux ont été atteints avec succès. Les enregistrements, qui étaient à l'origine informels et de qualité limitée, ont été enrichis et valorisés de manière significative. De plus, les outils de recherche développés, inexistants au départ, se révèlent particulièrement utiles, réduisant considérablement le temps nécessaire pour accéder à l'information.

1.2 Apports pour l'entreprise

La réalisation du projet Tech Event Expert a été bénéfique pour l'entreprise, à la fois sur le plan stratégique, technique et en termes de savoir-faire.

Sur le plan technique, le projet a atteint un niveau de maturité suffisant pour servir d'outil de démonstration auprès de potentiels clients. Le projet ouvre la possibilité de créer de nouvelles sources de revenus en transformant des enregistrements informels de conférences en contenus exploitables et structurés. Les outils développés pourraient permettre de réduire considérablement le temps nécessaire pour retrouver des informations ou générer des rapports techniques, offrant ainsi un avantage opérationnel et un service innovant que l'entreprise ne

proposait pas auparavant. À terme, ce système peut être étendu à une échelle industrielle, renforçant le positionnement compétitif de l'entreprise et créant de nouvelles opportunités économiques.

Ce projet a également permis d'expérimenter des technologies avancées et peu connues, telles que la diarisation des transcriptions audio, les modèles de vision pour l'extraction de données et l'utilisation des empreintes sonores humaines pour améliorer la précision et la contextualisation des informations. Ces expérimentations, couplées à l'usage de services de déploiement de modèles LLM en mode serverless, ont permis de tester les limites et les points forts des différentes approches dans le développement de systèmes RAG sur des documents non structurés.

Enfin, ce projet a enrichi le savoir-faire interne et ouvre des perspectives intéressantes pour l'avenir. La méthodologie et les outils développés pourraient être adaptés à d'autres types de contenus audio ou multimédia, offrant des opportunités dans des secteurs variés tels que l'éducation, la veille technologique ou la documentation industrielle. L'expérience acquise renforce la capacité de l'entreprise à innover et à proposer des services à forte valeur ajoutée.

2 Apports personnels

2.1 Compétences développées

Au cours de ce projet, j'ai eu l'opportunité de mener un travail de bout en bout, depuis la phase de conception jusqu'à la documentation finale. Cette expérience m'a permis de développer à la fois des compétences techniques et des compétences organisationnelles.

Sur le plan technique, j'ai acquis une connaissance approfondie des outils, bibliothèques et infrastructures nécessaires au développement de systèmes RAG. J'ai ainsi consolidé des compétences pratiques qui étaient peu exploitées lors de mon parcours académique, notamment l'utilisation de conteneurs Docker, la maîtrise de l'environnement AWS et de ses différents composants, ainsi que l'intégration d'outils modernes pour la gestion et l'optimisation de pipelines complexes.

Sur le plan organisationnel, le fait de piloter un projet complet m'a permis de renforcer ma capacité à planifier, structurer et coordonner efficacement les différentes étapes, tout en gardant une vision globale et cohérente du produit final.

Enfin, ce projet m'a donné une véritable vision entrepreneuriale de la science des données. J'ai pris conscience qu'il ne s'agit pas seulement de développer des modèles pour le plaisir technique, mais de mettre en œuvre les outils et méthodes disponibles pour répondre à de réels besoins concrets. Cette approche s'est d'ailleurs enrichie au contact des autres équipes, à travers les échanges sur leurs projets, leurs perspectives et leurs choix stratégiques. Elle m'a permis de replacer le développement technologique dans une dynamique plus large d'innovation et de création de valeur.

2.2 Impact sur le projet professionnel

Ce projet a eu un impact fort sur mon parcours personnel. Il m'a permis de réaliser que je ne souhaite pas uniquement développer des solutions techniques, mais aussi jouer un rôle plus

large dans la coordination, la vision et la stratégie des projets. J'ai découvert un véritable intérêt pour un profil plus managerial, où l'organisation, la communication et la prise de décision prennent autant d'importance que la partie technique.

Travailler sur un projet en intelligence artificielle m'a particulièrement plu, car ce domaine est en constante évolution, marqué par une énergie et une synergie uniques. J'ai pu constater à quel point l'IA crée des dynamiques collaboratives, favorise l'échange d'idées et ouvre la voie à des réseaux nationaux et internationaux denses et stimulants.

Cette expérience m'a donc conforté dans l'idée que je souhaite évoluer dans un rôle où je peux à la fois mobiliser mes compétences techniques et développer des qualités de leadership, de gestion de projet et de vision stratégique, afin de contribuer activement à l'innovation dans un domaine aussi vivant et porteur que l'intelligence artificielle.

3 Perspectives d'évolution

3.1 Suites possibles du projet

Pour la suite du projet, il serait pertinent d'initier une collaboration avec un client pilote susceptible d'être intéressé par ce produit. L'entreprise pourrait l'exploiter gratuitement, en échange de retours réguliers sur son utilisation et de propositions d'améliorations. Cette approche permettrait de maintenir un développement actif, en restant au plus près des besoins réels des utilisateurs finaux.

Une telle collaboration offrirait également l'opportunité de constituer une base de données plus vaste et plus diversifiée de fichiers de conférences, renforçant la robustesse et la valeur ajoutée du produit. Il restera toutefois nécessaire de définir clairement dans quel cadre ces fichiers pourront être réutilisés : uniquement pour le développement du produit au sein de l'entreprise partenaire, ou bien de manière élargie pour d'autres clients et cas d'usage.

Par ailleurs, le projet pourrait être poussé encore plus loin en l'appliquant à d'autres contextes, notamment les réunions internes d'entreprise ou d'autres événements professionnels. Grâce au même pipeline, il serait ainsi possible de traiter différents formats et sources de données, tout en gardant une architecture commune. Cela ouvrirait la voie à une multiplication des cas d'usage et à une meilleure valorisation de la solution dans des environnements variés, allant de la communication interne à la documentation technique et stratégique.

3.2 Améliorations techniques envisageables

Le projet présente encore plusieurs axes d'amélioration qui pourraient renforcer ses performances et sa pertinence. Un premier point concerne l'intégration initiale des fichiers média. Aujourd'hui, la qualité parfois médiocre des enregistrements audio peut constituer une limite dans la précision des transcriptions et des analyses. Il serait donc pertinent d'intégrer en amont un modèle de deep learning spécialisé dans l'amélioration du signal audio[20]. Ce type de modèle, capable de supprimer efficacement le bruit de fond et de rehausser la clarté des voix, pourrait considérablement améliorer la qualité de l'audio de départ. Dans notre cas, où les fichiers proviennent de contextes variés et souvent peu optimisés pour l'enregistrement, ce traitement préalable constituerait un atout majeur.

Un deuxième axe d'amélioration concerne le système de segmentation des textes. Bien qu'il soit efficace, son fonctionnement repose sur un nombre important d'itérations avec des modèles LLM. Cela soulève des enjeux non négligeables : d'une part en termes de coûts d'inférence, et d'autre part en matière d'écologie et d'économie d'énergie, du fait de la consommation importante liée à ces traitements répétés. Une piste prometteuse serait de développer ou de fine-tuner un modèle end-to-end capable de prendre un texte brut en entrée et de fournir directement en sortie une fonction de densité de probabilité utilisée pour guider la segmentation.

Pour l'architecture d'un tel modèle, il serait envisageable d'utiliser l'encodeur d'un LLM pré-entraîné comme base. Cela permettrait non seulement de bénéficier des performances offertes par des modèles ayant appris sur des corpus massifs et variés, mais aussi d'accélérer l'entraînement et d'améliorer la généralisation. Une telle approche pourrait réduire de manière significative les ressources nécessaires, tout en maintenant – voire en améliorant – la qualité des résultats.

Il serait également possible de pousser la réflexion sur la détection des locuteurs directement à partir de la transcription. Une approche intéressante pourrait consister à se concentrer uniquement sur les paroles de chaque personne et à n'extraire les données que lorsqu'un intervenant se présente lui-même. Cette méthode simplifierait grandement le travail du LLM pour l'étape d'extraction d'informations, en limitant le bruit et les ambiguïtés provenant des dialogues croisés.

De plus, la tâche d'extraction pourrait être plus modulable et structurée, comme nous l'avons évoqué dans la partie 1. Plutôt que d'aborder la transcription d'un seul bloc, il serait pertinent de diviser le travail en plusieurs étapes consécutives et plus simples. Par exemple :

- Vérifier si l'intervenant s'est présenté dans la transcription.
- Identifier si l'intervenant a mentionné certaines informations clés, telles que son prénom, son nom de famille et sa fonction.
- Extraire les informations trouvées une par une.
- Transformer les données extraites dans un format structuré, par exemple JSON, facilitant ainsi leur exploitation ultérieure.

3.3 Évaluation numérique

En raison des contraintes temporelles inhérentes à la durée du stage, il n'a pas été possible de procéder à une évaluation complète du système final. Or, l'évaluation constitue une étape cruciale pour valider la pertinence des choix techniques et quantifier la performance réelle du produit.

Dans une perspective de travaux futurs, il serait ainsi pertinent de mettre en place un protocole d'évaluation rigoureux. Celui-ci devrait porter dans un premier temps sur la qualité de la base vectorielle, en mesurant la capacité du moteur de recherche à retrouver des documents pertinents face à une requête donnée. Des métriques telles que le **Recall@k**, qui mesure la proportion de requêtes pour lesquelles un document pertinent figure parmi les k premiers résultats, ou encore le **Precision@k**, qui évalue la proportion de résultats pertinents parmi ces mêmes k documents, constituent des indicateurs standards. D'autres mesures comme le

Mean Reciprocal Rank (MRR), qui tient compte de la position du premier document pertinent, ou le nDCG (normalized Discounted Cumulative Gain), qui pondère la pertinence en fonction du rang, permettent également d'obtenir une vision plus fine de la qualité du système de recherche.

Dans un second temps, l'évaluation devrait s'étendre au système RAG dans son ensemble, en considérant non seulement la capacité de récupération, mais aussi la qualité des réponses générées. Plusieurs critères peuvent être mobilisés, tels que la fidélité ou *faithfulness*, qui vérifie que les réponses produites reposent effectivement sur les documents retrouvés, ainsi que la pertinence ou answer relevance, qui mesure l'adéquation de la réponse à la question posée. Pour des tâches fermées, des métriques objectives comme l'Exact Match ou le F1-score peuvent être utilisées, tandis que pour des cas plus ouverts, une évaluation humaine reste indispensable afin d'apprécier la clarté, l'exactitude et l'utilité perçue des réponses.

La mise en œuvre pratique de cette évaluation nécessiterait la constitution d'un jeu de test, comprenant des paires question-réponse servant de référence (ground truth). Les performances du système pourraient alors être comparées à ces données de référence, en s'appuyant à la fois sur des outils open source d'évaluation automatique tels que Ragas, LangChain Evaluation ou LlamaIndex Eval, et sur des retours qualitatifs d'utilisateurs. Une telle démarche offrirait une vision complète des forces et limites du système, et constituerait une étape clé pour assurer sa fiabilité dans un contexte d'utilisation réelle.

Table des figures

1.1	Graphique qui présente l'activité du Pôle Data and Process Intelligence	9
2.1	Graphique qui explique les difficultés du projet TEX par rapport aux précédents projets Doc2Value	12
2.2	Architecture initiale du flux des données du projet Tech Event Expert	14
2.3	Diagramme d'architecture de conception du projet Tech Event Expert	15
3.1	Graphique expliquant les raisons poussant au choix d'utilisation de whisper pour la transcription dans le projet Tech Event Expert	21
3.2	Un aperçu de l'interface web permettant d'ajouter des nouveaux fichiers média à la base de données	23
3.3	Rerésentation graphique du fonctionnement du processus d'intégration des nouveaux fichiers dans la base de données	24
3.4	Graphique illustrant l'algorithme de garantie d'unicité des fichiers dans la base de données par la méthode du hachage	25
3.5	Graphique expliquant la normalisation de format des fichiers multimédia avant tout traitement	26
3.6	Graphique expliquant le processus de transcription et de diarisation des fichiers audio employé dans le projet Tech Event Expert	28
3.7	Graphique explicatif de l'algorithme de segmentation utilisé lors du projet Tech Event Expert	29
3.8	Graphique expliquant l'algorithme de découpage textuel et de construction de la base de données du projet Tech Event Expert	30
3.9	Aperçu du moteur de recherche de démonstration utilisant la base de données développée dans le cadre du projet Tech Event Expert	32
3.10	Diagramme explicatif de l'agorithme envisagé pour la recherche d'un segment de texte et de son fichier média à partir d'une requête textuelle	34
A.1	Graphique de comparaison entre le système hybride et le système end-to-end pour la reconnaissance audio automatisé	93

Bibliographie

- [1] service groq. <https://groq.com>. Consulté le 13 août 2025.
- [2] Airbus. S. fleet performance + (skywise fleet performance). <https://aircraft.airbus.com/en/services/enhance/skywise-digital-solutions/skywise-fleet-performance>, 2025. Consulté en août 2025.
- [3] Sebastian Arnold, Rudolf Schneider, Philippe Cudré-Mauroux, Felix A. Gers, and Alexander Löser. Sector : A neural model for coherent topic segmentation and classification. *Transactions of the Association for Computational Linguistics*, 7 :169–184, 2019.
- [4] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0 : A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0 : A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 12449–12460, 2020.
- [6] Joe Barrow, Rishabh Jha, Ziyi Wang, Dongyeop Guo, Douglas W. Oard, Philip Resnik, and Mark Dredze. A joint model for document segmentation and segment labeling. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3133–3146, 2020.
- [7] Regina Barzilay and Michael Elhadad. Using lexical chains for text summarization. In *ACL Workshop on Intelligent Scalable Text Summarization*, pages 10–17, 1997.
- [8] Hervé Bredin. pyannote.audio 2.1 speaker diarization pipeline : Principle, benchmark, and recipe. In *Proceedings of Interspeech 2023*, pages 1983–1987, 2023.
- [9] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. Listen, attend and spell : A neural network for large vocabulary conversational speech recognition. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4960–4964. IEEE, 2016.

- [10] Freddy Y. Y. Choi. Advances in domain independent linear text segmentation. In *NAACL 2000*, pages 26–33, 2000.
- [11] Consensus. Introducing : Consensus gpt, your ai research assistant. <https://consensus.app/home/blog/introducing-researchgpt-by-consensus/>, jan 2024. Blog post, lancé en tant que « Consensus GPT » (anciennement ResearchGPT) le 10 janvier 2024.
- [12] Steven B. Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. In *IEEE Transactions on Acoustics, Speech, and Signal Processing*, volume 28, pages 357–366. IEEE, 1980.
- [13] Jacob Eisenstein and Regina Barzilay. Bayesian unsupervised topic segmentation. In *EMNLP 2008*, pages 334–343, 2008.
- [14] Hervé Bredin et al. pyannote.audio : neural building blocks for speaker diarization. <https://github.com/pyannote/pyannote-audio>, 2024. Release v3.0/3.1/3.3, benchmarks AMI, DIHARD, VoxConverse.
- [15] Yanick Fratantonio, Luca Invernizzi, Loua Farah, Thomas Kurt, Marina Zhang, Ange Albertini, François Galilee, Giancarlo Metitieri, Julien Cretin, Alexandre Petit-Bianco, David Tao, and Elie Bursztein. Magika : AI-Powered Content-Type Detection, 2024.
- [16] Michel Galley, Kathleen McKeown, Eric Fosler-Lussier, and Hongyan Jing. Discourse segmentation of multi-party conversation. In *ACL 2003*, pages 562–569, 2003.
- [17] Alex Graves. Sequence transduction with recurrent neural networks. In *Proceedings of the International Conference of Machine Learning (ICML) Workshop on Representation Learning*, 2012.
- [18] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification : Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, 2006.
- [19] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6645–6649. IEEE, 2013.
- [20] Sjoerd Groot, Qinyu Chen, Jan C. van Gemert, and Chang Gao. CleanUMamba : A Compact Mamba Network for Speech Denoising using Channel Pruning. In *Proceedings of the 2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, London, United Kingdom, 2025. IEEE.
- [21] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer :

- Convolution-augmented transformer for speech recognition. In *Proceedings of Interspeech*, pages 5036–5040, 2020.
- [22] Marti A. Hearst. Texttiling : Segmenting text into multi-paragraph subtopic passages. In *Computational Linguistics*, volume 23, pages 33–64, 1997.
- [23] Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, and Brian Kingsbury. Deep neural networks for acoustic modeling in speech recognition : The shared views of four research groups. *IEEE Signal Processing Magazine*, 29(6) :82–97, 2012.
- [24] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert : Self-supervised speech representation learning by masked prediction of hidden units. In *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, volume 29, pages 3451–3460, 2021.
- [25] IRT Saint-Exupéry. For program : Hybrid gen ai for critical processes. Site web de l’IRT Saint-Exupéry, 2024. Lancement du programme, investissement de plus de 90 M €, plus de 20 partenaires.
- [26] Omri Koshorek, Adir Cohen, Noam Mor, Michael Rotman, and Jonathan Berant. Text segmentation as a supervised learning task. In *Proceedings of NAACL-HLT (Volume 2 : Short Papers)*, pages 469–473, New Orleans, LA, 2018.
- [27] Michal Lukasik, Boris Dadachev, Shafiq Kong, Chen Zhuang, Antoine Miech, Yee Whye Teh, and Abir De. Text segmentation by cross-segment attention. In *Proceedings of EMNLP*, 2020.
- [28] Igor Malioutov and Regina Barzilay. Minimum cut model for spoken lecture segmentation. In *ACL 2006*, pages 25–32, 2006.
- [29] Modal. Top open-source stt models in 2025. <https://modal.com/blog/top-open-source-stt-models-2025>, 2025.
- [30] Tomohiro Nakatani, Yuya Fujita, Keisuke Kinoshita, and Marc Delcroix. Diaper : Perceiver-based attractors for end-to-end neural diarization. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024. Improved end-to-end diarization with Perceiver architecture.
- [31] NVIDIA. Multi-scale diarization decoder (msdd) model card. <https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/msdd>, 2023. NVIDIA NeMo diarization component.

- [32] NVIDIA. Sortformer streaming diarization model. https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/sortformer_streaming_diarization, 2023. DER and latency benchmarks on AMI-SDM.
- [33] NVIDIA. Nvidia parakeet : Fast and accurate speech recognition models. <https://developer.nvidia.com/blog/nvidia-parakeet-asr>, 2024.
- [34] NVIDIA. Nvidia parakeet model card. <https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2>, 2025.
- [35] OpenAI. Whisper large-v3 model card. <https://huggingface.co/openai/whisper-large-v3>, 2023.
- [36] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment : A simple data augmentation method for automatic speech recognition. In *Proceedings of Interspeech*, pages 2613–2617, 2019.
- [37] Rafael Pass. Collision-resistant hash functions and general digital signature scheme. <https://www.cs.cornell.edu/courses/cs6830/2009fa/scribes/lecture21.pdf>. Accessed : 2025-08-19.
- [38] Lawrence R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2) :257–286, 1989.
- [39] Alec Radford et al. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv :2212.04356*, 2022.
- [40] Google Research. Introducing google’s universal speech model. <https://ai.googleblog.com/2023/03/introducing-universal-speech-model.html>, 2023.
- [41] Meta AI Research. Seamless4t : Massively multilingual and multimodal machine translation. *Nature*, 2024.
- [42] Martin Riedl and Chris Biemann. Topictiling : A text segmentation algorithm based on lda. In *ACL 2012 Student Research Workshop*, pages 37–42, 2012.
- [43] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1) :43–49, 1978.
- [44] B. W. Silverman. *Density Estimation for Statistics and Data Analysis*. Monographs on Statistics and Applied Probability. Chapman and Hall, London, 1986.
- [45] Alessandro Solbiati et al. Unsupervised topic segmentation of meetings via contextual embeddings. In *Proceedings of Interspeech / or arXiv version*, 2021.

-
- [46] Swapna Somasundaran, Kun Qian, Young-Bum Song, Dapeng Guo, Eric Wallace, and Lei Li. Two-level transformer and auxiliary coherence modeling for improved topic segmentation. In *Proceedings of AAAI*, 2020.
 - [47] Masao Utiyama and Hitoshi Isahara. A statistical model for domain-independent text segmentation. In *ACL 2001*, pages 499–506, 2001.
 - [48] Jingdong Wang, Heng Tao Shen, Jingkuan Song, and Jianqiu Ji. Hashing for similarity search : A survey. *arXiv preprint arXiv :1408.2927*, 2014.
 - [49] Peilin Yang et al. 1. In *Proceedings of ACM SIGIR (Short/Workshop)—2025*, 2025. Position/Framework paper on LLM-based hierarchical segmentation.
 - [50] Steve J. Young. The HTK hidden markov model toolkit : Design and philosophy. In *Entropic Cambridge Research Laboratory*, 1994.
 - [51] Changlong Yu, Boliang Zhang, Yining Zhang, and Sudha Rao. Improving long document topic segmentation models with enhanced coherence modeling. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6147–6165, 2023.
 - [52] Yu Zhang et al. Usm : Scaling automatic speech recognition to 300+ languages. *arXiv preprint arXiv :2303.01037*, 2023.

Annexes

1 Documentation du projet réalisé

Ceci n'est qu'un aperçu de la documentation. La version finale prend la forme d'une page
web

Tech Event Expert

Version 0.0.1

EL MANNOUY Ayman, BENSOUDA Idriss

07 août 2025

Contents:

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

1.1 Structure du projet

Ce projet est organisé en **packages indépendants**, c'est-à-dire des composants autonomes, chacun situé dans un dossier distinct. Grâce à cette architecture modulaire, chaque composant peut être exécuté dans un contexte différent, voire sur une machine ou un serveur séparé.

Cependant, un ordre d'exécution est requis si l'on souhaite **ajouter des données à la base de données** :

1.1.1 Étapes du pipeline

1.1.1.1 1. data_files_selection

Ce module permet de fournir les **chemins des fichiers** à intégrer dans la base de données. Ces fichiers sont ensuite **envoyés au serveur**, en attente d'exécution des étapes suivantes du pipeline.

1.1.1.2 2. data_preprocessing

Ce package assure la **vérification** et la **normalisation** des fichiers fournis :

- Vérification que les fichiers sont dans le bon format et accompagnés des **métadonnées nécessaires**.
- Une couche de contrôle basée sur **Magika** (outil de Google) avait été initialement ajoutée. Elle prédisait l'extension d'un fichier à partir de son contenu binaire, permettant ainsi d'**éliminer les fichiers corrompus**. Cette étape a été supprimée par la suite, Magika s'étant révélé **peu fiable** dans certains cas.

Une fois les fichiers vérifiés, ils sont **convertis en format `.wav`**, un format audio **sans perte**, afin de **réduire la complexité** des traitements suivants et d'assurer une **uniformité** dans l'architecture des fichiers.

1.1.1.3 3. data_processing

Ce package contient le **noyau du traitement** :

- Transcription
- Diarisation
- Traitements techniques associés

L'utilisation d'un GPU est recommandée pour exécuter ce module de manière optimale.

1.1.2 Package expérimental

1.1.2.1 experimental

Ce package regroupe divers **outils expérimentaux** développés pour une éventuelle intégration au pipeline. Certains d'entre eux :

- manquent de maturité,
- nécessitent des réflexions supplémentaires,
- ou ont posé des **défis techniques** qui ont freiné leur intégration.

1.2 Prérequis

1.3 Configuration des bases de données

Le projet nécessite **trois serveurs de gestion de base de données** pour fonctionner correctement :

1. **Serveur S3** Stocke les fichiers **avant** et **après traitement**.
2. **Serveur MongoDB** Contient les **métadonnées saisies par les utilisateurs** ainsi que les **textes générés** par le pipeline de traitement.
3. **Serveur Milvus** Permet d'effectuer des **recherches rapides** dans la base de données des textes produits.

1.3.1 Configuration de MongoDB et Milvus

Pour mettre en place les serveurs **MongoDB** et **Milvus**, suivez les étapes ci-dessous :

1. Placez-vous dans le répertoire où vous souhaitez stocker les données.
2. Créez un fichier nommé `docker-compose.yml`.
3. Copiez le contenu suivant dans ce fichier :

```
version: '3.5'

services:
  etcd:
    container_name: milvus-etcd
    image: quay.io/coreos/etcd:v3.5.18
    environment:
      - ETCD_AUTO_COMPACTION_MODE=revision
      - ETCD_AUTO_COMPACTION_RETENTION=1000
      - ETCD_QUOTA_BACKEND_BYTES=4294967296
      - ETCD_SNAPSHOT_COUNT=50000
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/etcd:/etcd
    command: etcd -advertise-client-urls=http://etcd:2379 -listen-client-urls http://0.0.0.0:2379 --data-dir /etcd
    healthcheck:
      test: ["CMD", "etcdctl", "endpoint", "health"]
      interval: 30s
      timeout: 20s
      retries: 3

  minio:
    container_name: milvus-minio
    image: minio/minio:RELEASE.2023-03-20T20-16-18Z
    environment:
```

(suite sur la page suivante)

(suite de la page précédente)

```

    MINIO_ACCESS_KEY: minioadmin
    MINIO_SECRET_KEY: minioadmin
  ports:
    - "9001:9001"
    - "9000:9000"
  volumes:
    - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data
  command: minio server /minio_data --console-address ":9001"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 30s
    timeout: 20s
    retries: 3

standalone:
  container_name: milvus-standalone
  image: milvusdb/milvus:v2.5.9
  command: ["milvus", "run", "standalone"]
  security_opt:
    - seccomp:unconfined
  environment:
    ETCD_ENDPOINTS: etcd:2379
    MINIO_ADDRESS: minio:9000
  volumes:
    - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
    interval: 30s
    start_period: 90s
    timeout: 20s
    retries: 3
  ports:
    - "19530:19530"
    - "9091:9091"
  depends_on:
    - "etcd"
    - "minio"

mongodb:
  container_name: mongodb
  image: mongo:latest
  restart: always
  ports:
    - "27017:27017"
  environment:
    MONGO_INITDB_ROOT_USERNAME: admin
    MONGO_INITDB_ROOT_PASSWORD: admin
  volumes:
    - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/mongodb:/data/db

networks:
  default:

```

(suite sur la page suivante)

(suite de la page précédente)

```
name: milvus
```

4. Lancez les services en arrière-plan avec la commande suivante :

```
docker-compose up -d
```

1.3.2 Remarques sur la configuration par défaut

- **MongoDB**
 - Nom d'utilisateur : admin
 - Mot de passe : admin
 - Port : 27017
- **Milvus**
 - Aucun nom d'utilisateur ou mot de passe requis
 - Port : 19530

Il est bien entendu possible de **modifier cette configuration par défaut** directement dans le fichier `docker-compose.yml`.

1.4 Installation des dépendances

Chaque package du projet possède ses **propres dépendances**, qui peuvent différer de celles des autres modules. Néanmoins, un fichier `requirements.txt` est fourni à la racine du projet. Il permet d'installer l'ensemble des dépendances nécessaires pour exécuter **tous les packages**.

Pour installer ces dépendances, utilisez la commande suivante :

```
pip install -r requirements.txt
```

1.4.1 Remarque

Il est **fortement recommandé** de créer un **environnement virtuel** avant d'installer les dépendances, afin d'isoler les bibliothèques du projet et éviter les conflits avec d'autres environnements Python. Exemple avec `venv` :

```
python -m venv venv
source venv/bin/activate # Sous Unix/macOS
venv\Scripts\activate.bat # Sous Windows
```

1.5 Configuration de l'environnement (.env)

Pour assurer le bon fonctionnement du système, vous devez compléter le fichier `.env` avec les identifiants et informations de configuration nécessaires. Ce fichier contient les variables d'environnement permettant d'authentifier et de se connecter à différents services externes : MongoDB, Milvus, S3, Hugging Face et Groq.

Contenu attendu du fichier `.env` :

```
GROQ_API_KEY = # YOUR GROQ API TOKEN
HUGGING_FACE_API_TOKEN = # YOUR HUGGING FACE API TOKEN
MONGODB_USERNAME = # YOUR MONGODB SERVER USERNAME
MONGODB_PASSWORD = # YOUR MONGODB SERVER PASSWORD
MONGODB_HOST = # YOUR MONGODB SERVER HOST NAME
```

(suite sur la page suivante)

(suite de la page précédente)

```
MONGODB_PORT = # YOUR MONGODB SERVER HOST PORT
MILVUS_HOST = # YOUR MILVUS SERVER HOST
MILVUS_PORT = #YOUR MILVUS SERVER PORT
S3_BUCKET = # YOUR S3 BUCKETS NAME
S3_WORKING_DIR = # THE S3 PATH OF THE DIRECTORY CONTAINING DATA
```

1.5.1 Détails des variables

- **MongoDB** : Fournir les informations nécessaires une fois votre instance MongoDB correctement initialisée et accessible. Cela inclut :
 - MONGODB_USERNAME
 - MONGODB_PASSWORD
 - MONGODB_HOST
 - MONGODB_PORT
- **Milvus** : Remplir MILVUS_HOST et MILVUS_PORT avec les paramètres de votre serveur Milvus actif.
- **S3 (Amazon Simple Storage Service)** :
 - S3_BUCKET : nom du bucket à utiliser.
 - S3_WORKING_DIR : chemin vers le dossier de travail à l'intérieur du bucket contenant les données à traiter.
- **Hugging Face** : La variable HUGGING_FACE_API_TOKEN doit correspondre à un compte Hugging Face ayant **accepté les conditions d'utilisation** des modèles **pyannote**. Cela est indispensable pour accéder aux modèles suivants :
 - le modèle de **segmentation**
 - le modèle de **diarisation**
 - le modèle de **détection de parole superposée**L'acceptation des conditions peut se faire à l'adresse suivante : <https://huggingface.co/pyannote/models>
- **Groq** : Le service LLM sans serveur de **Groq** nécessite une clé d'API. La variable GROQ_API_KEY doit contenir une clé valide liée à votre compte Groq.

CHAPITRE 2

data_processing

2.1 data_files_selection package

Ce package contient un script dédié à l'envoi de fichiers à traiter depuis la machine locale vers le serveur. Il facilite le transfert manuel ou semi-automatisé des données en amont du processus de traitement, en garantissant leur mise à disposition dans l'environnement serveur dans les conditions attendues par le reste du système.

2.1.1 Submodules

2.1.2 data_files_selection.script module

Script de préparation de fichiers pour traitement sur S3

Ce script prend en entrée une liste de dossiers (*source_folders*) contenant chacun : - un fichier média (audio, vidéo) - un fichier de métadonnées associé (*meta-data.json*)

Il les envoie vers un emplacement spécifique sur un serveur S3, dans un dossier « en attente de traitement ». Ces fichiers seront ensuite traités automatiquement par d'autres composants du pipeline de traitement.

Avant d'exécuter ce script, il est impératif de configurer manuellement la variable *source_folders* avec les chemins **absolus** des dossiers à traiter. Assurez-vous que chaque dossier contient bien les deux fichiers requis.

2.2 data_preprocessing package

Ce module réalise les prétraitements essentiels avant toute étape ultérieure de traitement des fichiers multimédias. Il effectue une deuxième vérification rigoureuse de la structure des données pour s'assurer que chaque dossier contient un fichier audio ou vidéo accompagné de ses métadonnées dans un format attendu. Ensuite, le fichier média est converti dans un format standardisé .wav, choisi spécifiquement parce qu'il est non compressé, préservant ainsi l'intégralité de l'information sonore sans perte de qualité. Pour éviter toute redondance, le module garantit l'unicité des fichiers, aussi bien localement qu'au sein de la base de données distante.

Cette détection de doublons ne repose pas sur les noms des fichiers, mais sur le contenu binaire réel de chaque média : un hash MD5 est généré pour chaque fichier en lisant son contenu octet par octet. Ce hash agit comme une empreinte numérique unique ; deux fichiers identiques produiront le même hash, tandis que la moindre différence dans les données

générera un hash complètement différent, ce qui rend cette méthode fiable pour détecter les fichiers en double, même s'ils portent des noms différents.

2.2.1 Submodules

2.2.2 data_preprocessing.script module

Script de traitement de fichiers multimédias depuis un bucket S3.

Ce script effectue les étapes suivantes :

1. Téléchargement de fichiers multimédias et métadonnées depuis un répertoire *upload/* du bucket S3.
2. Vérification de la structure et du format des fichiers téléchargés.
3. Élimination des doublons basés sur les hashes MD5.
4. Conversion des fichiers multimédias vers un format audio normalisé *.wav*.
5. Téléversement des fichiers traités dans un répertoire *pre-processed/* du bucket S3.
6. Nettoyage local et suppression des fichiers originaux dans le répertoire *upload/* du bucket.

`data_preprocessing.script.convert_audio_to_wav(input_file, output_file)`

Calcule et retourne le hash MD5 du contenu d'un fichier.

Paramètres

filename (*str*) – Chemin vers le fichier dont on souhaite calculer le hash.

Renvoie

Représentation hexadécimale du hash MD5 du fichier.

Type renvoyé

str

`data_preprocessing.script.convert_media_file(file_path, output_file_path)`

Convertit un fichier média (audio ou vidéo) en format WAV en fonction de son extension.

Paramètres

— **file_path** (*str*) – Chemin vers le fichier média source (audio ou vidéo).

— **output_file_path** (*str*) – Chemin du fichier de sortie au format WAV.

Renvoie

None

`data_preprocessing.script.convert_video_to_wav(video_file, output_wav)`

Extrait la piste audio d'un fichier vidéo et la sauvegarde au format WAV.

Paramètres

— **video_file** (*str*) – Chemin vers le fichier vidéo source.

— **output_wav** (*str*) – Chemin du fichier de sortie au format WAV.

Renvoie

None

Lève

subprocess.CalledProcessError – Si la commande ffmpeg échoue.

`data_preprocessing.script.file_hash(filename)`

Génère un hash MD5 d'un fichier.

Paramètres

filename (*str*) – Chemin vers le fichier dont on veut calculer le hash.

Renvoie

Représentation hexadécimale du hash MD5 du contenu du fichier.

Type renvoyé

str

`data_preprocessing.script.load_media_folder(source_folder)`

Téléverse récursivement tout le contenu d'un dossier local vers un bucket S3 dans un sous-dossier unique généré dynamiquement.

Paramètres

source_folder (*str*) – Chemin vers le dossier local contenant les fichiers à téléverser.

Renvoie

None

`data_preprocessing.script.read_hash(filename)`

Calcule le hash MD5 d'un fichier en lisant son contenu binaire.

Paramètres

filename (*str*) – Chemin vers le fichier à analyser.

Renvoie

Hash MD5 sous forme hexadécimale, représentant de manière unique le contenu du fichier.

Type renvoyé

str

`data_preprocessing.script.save_hash(filename, hash_filename)`

Sauvegarde le hash MD5 d'un fichier dans un fichier texte.

Paramètres

— **filename** (*str*) – Chemin vers le fichier dont on veut générer le hash.

— **hash_filename** (*str*) – Chemin du fichier texte où le hash sera sauvegardé.

Renvoie

None

2.2.3 Module contents

2.3 data_processing package

2.3.1 Submodules

2.3.2 data_processing.script module

Ce script exécute les autres scripts du même package dans le bon ordre. Il permet d'effectuer tous les traitements nécessaires et de transférer les résultats vers les bases de données.

2.3.3 data_processing.script_filtirage module

Script de traitement de fichiers médias depuis un bucket S3.

Ce script se connecte à un bucket Amazon S3, télécharge les fichiers dans un dossier temporaire local, puis trie les fichiers en fonction de leur type (vidéo, audio, texte). Il prépare ainsi les fichiers pour un traitement ultérieur en les classant dans des sous-dossiers appropriés.

Étapes principales :

1. Téléchargement des fichiers depuis le dossier *pre-processed/* stocké dans le serveur S3.
2. Nettoyage de l'espace de travail local.
3. Classement des fichiers par type.

2.3.4 data_processing.script_transcription module

Ce script traite des fichiers audio et vidéo extraits d'un répertoire local, les segmente en morceaux, puis transcrit chaque segment à l'aide du modèle Whisper (via l'API Groq). Il reformate les horodatages pour les aligner avec le fichier original, nettoie les chevauchements entre segments, puis sauvegarde la transcription complète au format JSON. Le dossier temporaire de segments est supprimé à la fin du traitement.

`data_processing.script_transcription.format_transcriptions_timepoints`(*segments_transcriptions*,
chunk_duration=180,
stride=60)

Ajuste les timestamps des segments de transcription pour refléter leur position dans l’audio complet découpé avec recouvrement.

Chaque segment est décalé dans le temps en fonction de sa position, en tenant compte du recouvrement (*stride*) pour que les timestamps soient cohérents dans la transcription finale.

Paramètres

- **segments_transcriptions** (*list*) – Liste des listes de segments de transcription. Chaque segment contient des dicts avec “start”, “end”, “text”.
- **chunk_duration** (*int*) – Durée d’un segment audio (en secondes).
- **stride** (*int*) – Durée du recouvrement entre segments (en secondes).

Renvoie

Nouvelle liste de segments de transcription avec timestamps ajustés.

Type renvoyé

list

`data_processing.script_transcription.segment_audio`(*audio_filepath*, *output_dir='chunks/'*,
chunk_duration=180, *stride=60*)

Découpe un fichier audio en segments de durée fixe avec un recouvrement paramétrable. Cette fonction est principalement utilisée pour respecter la limite de taille des fichiers à envoyer à l’hébergeur des modèles de deep learning Groq. L’audio est découpé en morceaux, envoyés séparément au serveur, puis les résultats sont assemblés après traitement.

Paramètres

- **audio_filepath** (*str*) – Chemin vers le fichier audio à segmenter.
- **output_dir** (*str*, *optional*) – Dossier de sortie pour les segments audio. Par défaut « chunks/ ».
- **chunk_duration** (*int*, *optional*) – Durée d’un segment en secondes. Par défaut 180 secondes (3 minutes).
- **stride** (*int*, *optional*) – Recouvrement entre les segments en secondes. Par défaut 60 secondes (1 minute).

Renvoie

Un dictionnaire contenant le nombre total de segments générés sous la clé « segments_number ».

Type renvoyé

dict

`data_processing.script_transcription.strip_transcription_head`(*transcriptin*, *cut_length=30*)

Supprime les segments de transcription dont le début est avant une durée donnée.

Cette fonction filtre une transcription segmentée en excluant tous les segments dont le timestamp de début (*start*) est inférieur à *cut_length* secondes, afin de retirer les parties initiales non désirées.

Paramètres

- **transcription** (*list*) – Liste de segments de transcription, chaque segment étant un dict avec au moins les clés “start” et “end” (en secondes).
- **cut_length** (*int or float*) – Durée (en secondes) avant laquelle les segments sont supprimés.

Renvoie

Liste filtrée des segments dont le début est après *cut_length*.

Type renvoyé

list

`data_processing.script_transcription.strip_transcription_tail`(*transcriptin*,
total_transcription_duration=180,
cut_length=30)

Supprime les segments de transcription qui commencent après une certaine durée depuis la fin.

Cette fonction filtre une transcription segmentée en excluant tous les segments dont le timestamp de début (*start*) est supérieur à la durée totale moins une marge *cut_length*. Cela permet de retirer les parties finales non désirées.

Paramètres

- **transcription** (*list*) – Liste de segments de transcription, chaque segment étant un dict avec au moins les clés “start” et “end” (en secondes).
- **total_transcription_duration** (*int or float*) – Durée totale (en secondes) de la transcription.
- **cut_length** (*int or float*) – Durée (en secondes) à découper à la fin de la transcription.

Renvoie

Liste filtrée des segments dont le début est avant la coupure à la fin.

Type renvoyé

list

`data_processing.script_transcription.strip_transcriptions(segments_transcriptions, chunk_duration=180, cut_length=30)`

Nettoie les transcriptions segmentées en supprimant les chevauchements aux débuts et fins des segments.

Cette fonction traite une liste de transcriptions segmentées issues d’un découpage audio. Pour éviter les chevauchements et redondances dans la transcription globale : - Le premier segment a sa queue (fin) tronquée. - Les segments intermédiaires ont leur tête (début) et queue (fin) tronquées. - Le dernier segment a sa tête (début) tronquée. Si un seul segment est présent, il est retourné sans modification.

Paramètres

- **segments_transcriptions** (*list*) – Liste de listes de segments de transcription. Chaque segment de transcription est une liste de dicts avec les clés “start”, “end”, “text”.
- **chunk_duration** (*int*) – Durée d’un segment audio (en secondes).
- **cut_length** (*int*) – Durée (en secondes) à retirer au début ou à la fin des segments.

Renvoie

Liste des transcriptions segmentées nettoyées.

Type renvoyé

list

`data_processing.script_transcription.transcribe_audio(filepath)`

Transcrit un fichier audio en texte avec horodatage des segments.

Utilise le client Whisper pour générer une transcription détaillée, segmentée avec les timestamps de début et fin de chaque segment.

Paramètres

filepath (*str*) – Chemin vers le fichier audio à transcrire.

Renvoie

Liste de segments transcrits.

Chaque segment est un dictionnaire contenant : - “start” (float) : début du segment en secondes, - “end” (float) : fin du segment en secondes, - “text” (str) : texte transcrit du segment.

Type renvoyé

list of dict

`data_processing.script_transcription.transcribe_segments(chunks_dir, segmentation_data)`

Transcrit une série de segments audio stockés dans un répertoire.

Parcourt les fichiers audio segmentés numérotés séquentiellement, utilise la fonction *transcribe_audio* pour chaque segment, et collecte les transcriptions dans une liste.

Paramètres

- **chunks_dir** (*str*) – Chemin vers le dossier contenant les segments audio (fichiers .wav).
- **segmentation_data** (*dict*) – Dictionnaire contenant au moins la clé “segments_number” indiquant le nombre total de segments à transcrire.

Renvoie

Liste des transcriptions pour chaque segment.

Chaque élément est une liste de dictionnaires représentant les segments transcrits avec leurs timestamps et texte (format retourné par *transcribe_audio*).

Type renvoyé

list

2.3.5 data_processing.script_diarization_chunking module

Ce script traite des fichiers média (audio/vidéo) en plusieurs étapes :

1. Il récupère les fichiers audio et leurs transcriptions depuis des répertoires locaux.
2. Il applique une diarisation vocale pour identifier les différents locuteurs dans les fichiers audio.
3. Il aligne les segments de la transcription avec les locuteurs identifiés.
4. Il fusionne les segments consécutifs du même locuteur pour créer une transcription plus cohérente.
5. Il segmente la transcription fusionnée en blocs thématiques en utilisant un LLM pour détecter les points de séparation.
6. Il applique une méthode statistique (KDE) pour consolider ces points de séparation.
7. Il exporte les résultats en fichiers JSON, structurés par segments de discussions.

Ce processus permet une meilleure structuration de discussions longues (comme des conférences), en séparant les interventions pertinentes par locuteur et par thème.

`data_processing.script_diarization_chunking.LLM_seperator_points(radical_chunks,
iterations_per_chunk=30)`

Utilise un modèle LLM pour identifier des points de séparation thématiques dans des chunks de transcription.

Paramètres

- **radical_chunks** (*list*) – Liste de chunks de transcription (listes de segments).
- **iterations_per_chunk** (*int*) – Nombre d’itérations par chunk pour obtenir des propositions variées.

Renvoie

Liste des points de séparation temporels extraits par le modèle.

Type renvoyé

list

`data_processing.script_diarization_chunking.align_transcription_with_diarization(transcription,
diari-
zed_segments)`

Aligne chaque segment de transcription avec le locuteur correspondant selon la diarisation.

Paramètres

- **transcription** (*list of dict*) – Liste de segments de transcription, chaque segment est un dict contenant au minimum les clés “start” (début en secondes), “end” (fin en secondes), et “text” (texte transcrit).
- **diarized_segments** (*list of tuples*) – Liste des segments de diarisation sous la forme (start, end, speaker), où “start” et “end” sont les temps en secondes, et “speaker” est l’identifiant du locuteur.

Renvoie

Liste des segments transcrits enrichis avec la clé “speaker” correspondant au locuteur assigné. Chaque dict contient “start”, “end”, “text” et “speaker”.

Type renvoyé

list of dict

Description :

La fonction calcule le chevauchement temporel entre chaque segment de transcription et les segments de diarisation. Le locuteur est attribué au segment de transcription selon la durée maximale de chevauchement avec les segments diarisés.

`data_processing.script_diarization_chunking.diarize_audio(audio_path)`

Effectue la diarisation des locuteurs sur un fichier audio donné.

Paramètres

audio_path (*str*) – Chemin vers le fichier audio à analyser.

Renvoie

Liste des segments détectés sous la forme (start, end, speaker),

où “start” et “end” sont les temps en secondes du segment, et “speaker” est l’identifiant du locuteur.

Type renvoyé

list of tuples

Description :

La fonction utilise le modèle pré-entraîné « pyannote/speaker-diarization-3.1 » pour identifier les segments temporels attribués à chaque locuteur dans l’audio. L’exécution se fait sur GPU si disponible, sinon sur CPU.

`data_processing.script_diarization_chunking.merge_continuous_speaker_segments(diarized_transcription, max_talk_duration=45)`

Fusionne les segments consécutifs d’un même locuteur en un seul segment, à condition que la durée totale ne dépasse pas une limite donnée.

Paramètres

— **diarized_transcription** (*list of dict*) – Liste de segments transcrits avec locuteur, chaque segment contenant au minimum les clés “start”, “end”, “speaker”, et “text”.

— **max_talk_duration** (*float*) – Durée maximale (en secondes) d’un segment fusionné. Si la durée totale dépasse cette valeur, la fusion s’arrête.

Renvoie

Liste de segments fusionnés où les segments consécutifs du même locuteur ont été combinés en un seul segment, avec le texte concaténé et les temps ajustés.

Type renvoyé

list of dict

Description :

La fonction parcourt la liste des segments et fusionne les segments consécutifs si le locuteur est identique et si la durée cumulée reste inférieure à max_talk_duration. Elle retourne une nouvelle liste avec les segments fusionnés.

`data_processing.script_diarization_chunking.radical_transcription_chunks(merged_transcription, radical_chunking_duration=600, overlap=0.75)`

Découpe une transcription fusionnée en segments (« chunks ») radicaux de durée fixe avec chevauchement.

Paramètres

— **merged_transcription** (*list of dict*) – Liste des segments fusionnés, chaque segment devant contenir au moins les clés “start” et “end” (en secondes).

— **radical_chunking_duration** (*int, optional*) – Durée maximale (en secondes) de chaque chunk. Par défaut 600 secondes (10 minutes).

— **overlap** (*float, optional*) – Pourcentage de chevauchement entre deux chunks consécutifs, exprimé entre 0 (pas de chevauchement) et 1 (100%). Par défaut 0.75 (75%).

Renvoie

Liste de chunks, chaque chunk étant une liste de segments dont les temps se situent dans l'intervalle du chunk.

Type renvoyé

list of list of dict

Description :

La fonction crée des segments radicaux en avançant sur la transcription fusionnée par intervalles de durée fixe avec un chevauchement donné. Chaque chunk contient les segments dont les temps sont entièrement compris dans l'intervalle de chunk.

data_processing.script_diarization_chunking.statistical_seperator_points(points)

Identifie des points de séparation temporels à partir d'une liste de points, en utilisant une estimation de densité par noyau (KDE) pour détecter les maxima locaux.

Paramètres

points (*list* ou *np.array*) – Liste de points temporels (en secondes).

Renvoie

Coordonnées x des maxima locaux détectés dans la densité KDE.

Type renvoyé

np.array

2.3.6 data_processing.script_vectorisation module

Ce script automatise le traitement de fichiers multimédias (audios/vidéos) et de leurs métadonnées.

Il suit les étapes suivantes :

1. Connexion à MongoDB, Milvus et Amazon S3.
2. Chargement et encodage de transcriptions en vecteurs sémantiques.
3. Enregistrement des vecteurs dans Milvus (base vectorielle).
4. Annotation des textes avec des étiquettes de haut niveau via l'API Groq.
5. Sauvegarde des métadonnées enrichies dans MongoDB.
6. Déplacement local des fichiers traités.
7. Téléversement sur S3 des résultats finaux.
8. Nettoyage des fichiers traités localement et sur S3.

data_processing.script_vectorisation.get_chunk_label(text : str) → str

Génère un label de catégorie haute-niveau approprié pour un texte donné en interrogeant un modèle de langage.

Paramètres

text (*str*) – Le texte à catégoriser.

Renvoie

Le label de catégorie le plus adapté au contenu du texte, sous forme de texte simple.

Type renvoyé

str

Description :

La fonction construit une requête pour demander au modèle LLaMA 3.3 70B d'identifier la catégorie principale du texte fourni. Elle utilise une requête explicite demandant uniquement le label sans information additionnelle.

2.4 experimental package

2.4.1 Subpackages

2.4.1.1 experimental.aglorthmic_speaker_data_extractor package

Submodules

experimental.aglorthmic_speaker_data_extractor.script module

Script Python pour extraire les informations des intervenants (conférenciers) à partir d'un fichier HTML provenant d'une conférence (ici WATS - World Aviation Training Summit).

Le script :

1. Charge un fichier HTML.
2. Nettoie le texte brut en supprimant les balises inutiles.
3. Formate et nettoie le contenu textuel.
4. Envoie ce contenu à un modèle de langage via l'API Groq pour extraire les données des intervenants.
5. Affiche les données extraites au format JSON.

Les données extraites pour chaque intervenant incluent : - Prénom - Nom - Affiliation (si mentionnée)

2.4.1.2 experimental.algorithmic_text_detection package

Submodules

experimental.algorithmic_text_detection.script module

Script : text_detection_opencv.py

Objectif :

Détecter automatiquement les zones contenant du texte dans une image en utilisant des techniques de traitement d'image avec OpenCV.

Paramètres à configurer :

- image_path : chemin vers l'image d'entrée à analyser
- Les dossiers de sortie doivent exister (ici « grayscaled image/ »)

Étapes principales :

1. Lecture et conversion en niveaux de gris
2. Floutage pour réduire le bruit
3. Seuillage adaptatif (binarisation) pour isoler le texte
4. Dilatation pour regrouper les caractères en blocs de texte
5. Détection des contours et encadrement des zones de texte
6. Sauvegarde des étapes intermédiaires pour visualisation

Résultat :

Une image avec les zones de texte détectées entourées de rectangles verts.

2.4.1.3 experimental.inter_files_diarization package

Submodules

experimental.inter_files_diarization.construct_bdd_vectors_from_audios module

Ce script parcourt un ensemble de fichiers audio (*media.wav*) situés dans un dossier donné. Pour chaque fichier, il extrait des embeddings de locuteurs (représentations vectorielles) à l'aide d'une fonction dédiée, puis insère ces embeddings dans une base de données MongoDB.

Étapes principales :

1. Chargement des variables d'environnement (jeton Hugging Face, accès MongoDB)
2. Recherche des fichiers audio dans une structure de dossiers
3. Extraction des embeddings via une fonction externe
4. Transformation des données au format MongoDB
5. Insertion des documents dans la base de données

`experimental.inter_files_diarization.find_speaker_in_bdd_by_voice` module

Script de comparaison d'embeddings vocaux avec une base de données MongoDB.

Ce script extrait les embeddings d'un locuteur à partir d'un fichier audio, les compare aux embeddings existants dans la base de données MongoDB, calcule les similarités cosinus et retourne les meilleurs résultats triés par similarité moyenne.

Utilisation :

- Spécifiez le chemin du fichier audio (*media_file*)
- Spécifiez l'ID du locuteur (*Speaker*)

`experimental.inter_files_diarization.find_speaker_in_bdd_by_voice.compare_vectors_efficient_batching(query_vectors, collection)`

Version optimisée en mémoire de la comparaison, par lots.

Paramètres

- **query_vectors** – Vecteurs à comparer
- **collection** – Collection MongoDB
- **batch_size** – Taille des lots pour le traitement

Renvoie

Similarités moyennes par (file_id, speaker_id)

Type renvoyé

dict

`experimental.inter_files_diarization.find_speaker_in_bdd_by_voice.compare_vectors_with_database(query_vectors, collection)`

Compare les vecteurs de requête avec ceux de la base de données. Retourne les similarités groupées par fichier et locuteur.

Paramètres

- **query_vectors** – Liste de vecteurs à comparer
- **collection** – Collection MongoDB contenant les embeddings

Renvoie

Dictionnaire des similarités moyennes par (file_id, speaker_id)

Type renvoyé

dict

`experimental.inter_files_diarization.find_speaker_in_bdd_by_voice.get_top_matches(averaged_similarities, top_k=10)`

Récupère les top K correspondances les plus similaires.

Paramètres

- **averaged_similarities** – Dictionnaire retourné par la fonction de comparaison
- **top_k** – Nombre de résultats à retourner

Renvoie

Liste triée des top K similarités

Type renvoyé

list

experimental.inter_files_diarization.vectorisation_utils module

Script d'extraction d'empreintes vocales à partir de fichiers audio

Ce script utilise PyAnnote pour :

1. Identifier les locuteurs présents dans un fichier audio (diarisation).
2. Détecter les segments de parole se chevauchant.
3. Extraire les segments de parole non chevauchés pour chaque locuteur.
4. Générer des embeddings (vecteurs représentatifs) pour ces segments propres.

Les embeddings peuvent être utilisés pour reconnaître ou regrouper les locuteurs. Ce pipeline est compatible GPU pour accélérer le traitement.

`experimental.inter_files_diarization.vectorisation_utils.get_embeddings_list(audio_file)`

Extrait des embeddings audio associés à chaque locuteur à partir d'un fichier audio.

Cette fonction effectue les étapes suivantes :

1. Charge l'audio et récupère sa durée totale.
2. Effectue la diarisation pour détecter les tours de parole par locuteur.
3. Détecte les segments de parole où plusieurs personnes parlent en même temps.
4. Supprime les segments de parole chevauchés.
5. Conserve uniquement les segments non-chevauchés attribués à un seul locuteur.
6. Applique une fenêtre glissante sur chaque segment pour calculer les embeddings audio.
7. Retourne la liste des embeddings avec le locuteur correspondant.

Paramètres

audio_file (*str*) – Chemin vers le fichier audio d'entrée.

Renvoie

Une liste de dictionnaires, chacun contenant :

- "speaker" (*str*) : L'identifiant du locuteur (par exemple « SPEAKER_00 »).
- "embedding" (*torch.Tensor*) : Le vecteur d'embedding audio associé.

Type renvoyé

List[dict]

Exemple :

```
>>> embeddings = get_embeddings_list("chemin/vers/audio.wav")
>>> print(embeddings[0]['speaker']) # 'SPEAKER_01'
>>> print(embeddings[0]['embedding'].shape) # torch.Size([512]) ou similaire
```

2.4.1.4 experimental.vision_model_extractor_of_speaker_data package**Submodules****experimental.vision_model_extractor_of_speaker_data.script module**

Ce script utilise une API de vision (Ollama) avec le modèle "llama3.2-vision" pour analyser une image (par exemple une page web d'une conférence aéronautique) et tente d'en extraire la liste des intervenants mentionnés.

2.4.1.5 experimental.detection_speakers_par_des_LLMs package

Submodules

experimental.detection_speakers_par_des_LLMs.script module

Ce script télécharge des transcriptions avec métadonnées depuis une base de données MongoDB, analyse chaque transcription pour extraire les informations personnelles des intervenants (nom, prénom, métier, affiliation) à l'aide d'un modèle LLM hébergé sur Groq, puis stocke les résultats vérifiés dans une autre collection MongoDB.

Il utilise également une vérification automatique pour s'assurer que les informations extraites sont bien justifiées par les citations dans la transcription, et que celles-ci proviennent uniquement de l'intervenant concerné.

`experimental.detection_speakers_par_des_LLMs.script.extract_data(speaker, transcript)`

Extrait des informations personnelles mentionnées par un locuteur donné dans une transcription, en s'appuyant sur un LLM.

Les données extraites sont : prénom, nom de famille, métier, et affiliation. Le modèle doit également fournir des citations exactes comme preuve.

Paramètres

- **speaker** (*str*) – Nom ou identifiant du locuteur.
- **transcript** (*str*) – Texte dit uniquement par ce locuteur.

Renvoie

Dictionnaire structuré contenant les informations extraites et leurs preuves.

Type renvoyé

dict

`experimental.detection_speakers_par_des_LLMs.script.extract_speakers_data(transcript)`

Extracts validated personal information for each speaker in a structured transcript.

This function : - Identifies speakers (e.g., « SPEAKER_01 ») in the transcript. - Extracts personal data mentioned by each speaker (first name, family name, job, affiliation). - Validates each extracted field using quoted proof and a separate verification step.

Paramètres

- **transcript** (*str*) – The full transcript containing speaker labels and dialogue.

Renvoie

A list of dictionaries. Each dictionary contains :

- **speaker** (*str*) : Speaker ID
- **data** (**List[dict]**) : **List of data entries, each with :**
 - **title** (*str*) : « first_name », « family_name », « job », or « affiliation »
 - **content** (*str*) : The extracted value
 - **proofs** (**List[str]**) : Supporting quotes from the transcript
 - **confidence** (*float*) : Confidence score from 0 to 1

Type renvoyé

List[dict]

Exemple

```
>>> transcript = "SPEAKER_01: Je m'appelle Jean Dupont, je suis ingénieur chez Airbus."
>>> extract_speakers_data(transcript)
[
  {
    "speaker": "SPEAKER_01",
    "data": [
```

(suite sur la page suivante)

(suite de la page précédente)

```

        {
            "title": "first_name",
            "content": "Jean",
            "proofs": ["Je m'appelle Jean Dupont"],
            "confidence": 0.98
        },
        ...
    ]
}
]
```

```
experimental.detection_speakers_par_des_LLMs.script.verify_extraction(speaker,
                                                                    information_title,
                                                                    information, proofs,
                                                                    transcript)
```

Vérifie si l'information extraite d'une transcription est bien justifiée par des preuves attribuables au locuteur donné.

Paramètres

- **speaker** (*str*) – Nom ou identifiant du locuteur.
- **information_title** (*str*) – Titre de l'information extraite (e.g., "job").
- **information** (*str*) – Valeur extraite.
- **proofs** (*list[str]*) – Citations exactes servant de preuve.
- **transcript** (*str*) – Transcription complète des propos du locuteur.

Renvois

Résultat de la vérification ({ « result » : « Yes »/ « No », « confidence » : float}).

Type renvoyé

dict

2.5 utils package

Ce package est une bibliothèque locale, développée spécifiquement dans le cadre de ce projet. Il centralise un ensemble de fonctionnalités communes, utiles à plusieurs composants ou modules du projet. L'objectif de cette bibliothèque est de favoriser la réutilisabilité du code, de mutualiser certaines logiques partagées, et d'assurer une meilleure organisation et maintenance de l'ensemble du projet.

2.5.1 Submodules

2.5.2 utils.file_queueing_for_processing module

Ce module regroupe les outils nécessaires à l'envoi des fichiers, ainsi que de leurs métadonnées, vers le serveur S3, dans le répertoire désigné comme « en attente de traitement ». Il constitue l'une des premières étapes du pipeline, en déposant les éléments à traiter dans un emplacement surveillé par les composants suivants du système, qui se chargent ensuite de leur traitement.

```
utils.file_queueing_for_processing.load_media_folder(source_folder)
```

Fonction de téléversement vers AWS S3.

Elle prend un dossier local contenant un fichier média et un fichier de métadonnées, et le copie dans un sous-dossier unique de S3 dans le répertoire *upload/*.

Le sous-dossier est automatiquement nommé avec un horodatage et un identifiant aléatoire, afin d'éviter tout écrasement de fichiers et garantir la traçabilité.

Paramètres

source_folder (*str*) – Chemin vers le dossier local contenant les fichiers à téléverser.

`utils.file_queueing_for_processing.process_folders(source_folders)`

Fonction de traitement en lot des dossiers médias.

Pour chaque dossier dans *source_folders*, cette fonction effectue deux étapes :

1. **Vérifie que le dossier contient exactement deux fichiers :**
 - un fichier *meta-data.json*
 - un fichier média (audio, vidéo ou texte) avec une extension valide.
2. Si le dossier est valide, il est téléversé dans un sous-dossier unique du bucket S3, sous le répertoire *upload/*.

Toute erreur de validation dans un dossier arrête l'exécution du programme.

Paramètres

source_folders (*list[str]*) – Liste des chemins vers les dossiers à traiter.

`utils.file_queueing_for_processing.verify_and_upload_folder(source_folder)`

+ Fonction combinée :

- **Vérifie que le dossier source respecte un format strict :**
 1. exactement 2 fichiers,
 2. un fichier nommé *meta-data.json*,
 3. un fichier média audio/vidéo/texte avec extension valide.
- Si le dossier est valide, le transfère vers le dossier S3 « en attente de traitement ».

Paramètres

source_folder (*str*) – Chemin vers le dossier local à vérifier et téléverser.

2.5.3 `utils.file_verification_functions` module

OBJECTIF GÉNÉRAL :

Ce module fournit des fonctions de vérification qui s'assurent que les dossiers contenant des médias à téléverser sont bien formatés.

Ces dossiers doivent obligatoirement :

1. Contenir exactement deux fichiers,
2. Contenir un fichier nommé "meta-data.json",
3. Contenir un fichier média (audio, vidéo, ou texte), à l'exclusion de tout autre type de fichier.

Si l'une de ces conditions échoue, une exception est levée pour empêcher le traitement automatique ou l'envoi sur S3.

exception `utils.file_verification_functions.WrongInputFormatError`

Bases : `Exception`

Exception personnalisée utilisée pour signaler une erreur de format de dossier non conforme aux règles attendues.

`utils.file_verification_functions.folder_contains_2_files(source_folder)`

Vérifie que le dossier contient exactement 2 fichiers.

Cette contrainte est importante pour garantir qu'il y a un et un seul fichier média, accompagné d'un fichier de métadonnées.

Paramètres

source_folder (*str*) – Chemin vers le dossier à vérifier.

Renvoie

True si le dossier contient exactement 2 fichiers, sinon False.

Type renvoyé

bool

`utils.file_verification_functions.folder_contains_media_file(source_folder)`

Vérifie qu'un fichier média valide est présent, en plus du fichier de métadonnées.

Le fichier peut être : - un média vidéo (ex : .mp4) - un média audio (ex : .mp3) - ou un fichier texte (ex : .txt, .json sauf "meta-data.json")

Paramètres**source_folder** (*str*) – Chemin vers le dossier à vérifier.**Renvoie**

True si un fichier média valide est présent, sinon False.

Type renvoyé

bool

`utils.file_verification_functions.folder_contains_metadata(source_folder)`

Vérifie que le fichier de métadonnées "meta-data.json" est bien présent dans le dossier.

Ce fichier est essentiel pour fournir les informations d'encodage, description, langue, etc.

Paramètres**source_folder** (*str*) – Chemin vers le dossier à vérifier.**Renvoie**

True si le fichier "meta-data.json" est présent, sinon False.

Type renvoyé

bool

`utils.file_verification_functions.verify_folder(source_folder)`

Valide la structure d'un dossier avant traitement.

Elle combine toutes les vérifications : - exactitude du nombre de fichiers, - présence du fichier "meta-data.json", - présence d'un fichier média autorisé.

En cas d'erreur, lève une exception descriptive pour faciliter le diagnostic par un utilisateur ou un système de logs.

Paramètres**source_folder** (*str*) – Chemin vers le dossier local à valider.**Lève****WrongInputFormatError** – Si le dossier ne respecte pas les contraintes requises.

2.6 interface_graphique_pour_integration_des_fichiers_package

Page web de démonstration d'un outil de téléversement de fichiers multimédias avec métadonnées.

Cette démonstration illustre un outil développé dans le cadre de ce projet, permettant à un utilisateur d'ajouter des fichiers audio ou vidéo au serveur, accompagnés de métadonnées. Certaines métadonnées sont obligatoires (ex. : titre, langue, type de média), tandis que d'autres sont facultatives et peuvent être ajoutées librement.

L'outil vérifie le format des fichiers transmis et communique avec la base de données pour enregistrer les informations fournies. L'ensemble des traitements techniques (vérification des fichiers, accès base de données, structuration des métadonnées, etc.) s'appuie principalement sur le paquet local *utils*.

Cette page sert uniquement à démontrer le fonctionnement de cet outil, dans un cadre simple et reproductible.

2.7 moteur_de_recherche_interface_web package

Page web de démonstration d'un moteur de recherche multimédia basé sur Milvus, Django et Amazon S3.

Cette démonstration utilise les outils développés dans le cadre de ce projet pour permettre la recherche et la lecture de fichiers audio et vidéo à partir d'une base de données vectorielle.

Fonctionnement :

1. L'utilisateur saisit une requête textuelle dans le champ de recherche.
2. La requête est vectorisée, puis une recherche de similarité est effectuée dans la base Milvus préexistante.
3. Les identifiants retournés par Milvus sont utilisés pour interroger la base de données Django, afin de récupérer les métadonnées et les informations nécessaires.
4. Les fichiers multimédias correspondants (audio ou vidéo) sont localisés sur un serveur Amazon S3 et intégrés dans la page.
5. L'utilisateur peut ensuite consulter les résultats et lire les fichiers via un lecteur intégré.

3.1 Chatbot module

Chatbot Module

This module is designed to handle user queries, process audio files, and generate responses using a large language model (LLM).

It includes functionalities for embedding queries, searching a database, and synthesizing responses based on the retrieved data.

This code is part of a larger project that includes audio transcription, synthesis, database management, and this chatbot.

This module uses the DBEmbed module to embed queries, search the database, retrieve and clusterize relevant documents.

`Chatbot.Chatbot.consistent(docs : str, outputs : dict) → bool`

Checks if the outputs are consistent with the provided documents.

Paramètres

- **docs** (str) – The documents to check consistency against.
- **outputs** (dict) – The outputs containing the answer and reasoning.

Renvoie

True if the outputs are consistent with the documents, otherwise a score indicating the level of consistency.

Type renvoyé

bool

`Chatbot.Chatbot.generate_rationale(docs : str, query : str)`

Generates a response and rationale based on the provided documents and query.

Paramètres

- **docs** (str) – The documents to base the response on.
- **query** (str) – The query to answer.

Renvoie

A tuple containing the response and rationale.

Type renvoyé

tuple

Chatbot.Chatbot.**local_llm_as_judge**(*example : dict*) → dict

Evaluates the example using a local LLM as a judge for hallucination.

Paramètres

— **example** (*dict*) – The example containing inputs, outputs, context, and reference outputs.

Renvoie

A dictionary containing the hallucination score.

Type renvoyé

dict

Chatbot.Chatbot.**process_subset**(*e, query, evidence, outputs*)

Processes a subset of evidence to generate a response and evaluate it.

Paramètres

— **e** (*str*) – The name of the subset.

— **query** (*str*) – The query to answer.

— **evidence** (*dict*) – A dictionary containing evidence for each subset.

— **outputs** (*dict*) – A dictionary to store the outputs for each subset.

Renvoie

A tuple containing the name of the subset and its corresponding output.

Type renvoyé

tuple

Chatbot.Chatbot.**understandable**(*inputs : dict, outputs : dict*) → bool

Evaluates the understandability of the answer based on the question and answer provided.

Paramètres

— **inputs** (*dict*) – The inputs containing the question.

— **outputs** (*dict*) – The outputs containing the answer.

Renvoie

A score indicating the understandability of the answer, ranging from 0 to 4.

Type renvoyé

str

Chatbot.Chatbot.**valid_reasoning**(*inputs : dict, outputs : dict*) → bool

Validates the reasoning provided in the outputs against the inputs.

Paramètres

— **inputs** (*dict*) – The inputs to validate against.

— **outputs** (*dict*) – The outputs containing the reasoning to validate.

Renvoie

True if the reasoning is valid, otherwise a message indicating the issue.

Type renvoyé

bool

3.2 DBEmbed module

Chatbot.DBEmbed.**cluster_results**(*k : int, results, embeddings, files*)

Clusters the results using KMeans and reduces dimensions with PCA.

Paramètres

— **k** (*int*) – The number of clusters to form.

— **results** (*list*) – A list of dictionaries containing formatted results from the database.

— **embeddings** (*list*) – A list of embeddings corresponding to the results.

— **files** (*list*) – A list of media file names corresponding to the results.

Renvoie

A dictionary where keys are subset indices and values are lists of media file

Type renvoyé

subsets (dict)

Chatbot.DBEmbed.**connect_to_db**()

Connects to MongoDB and Milvus, initializes the SentenceTransformer model.

Chatbot.DBEmbed.**format_files**(*subsets, formatted_results*)

Formats the results into a dictionary with media file names as keys and chunk texts as values.

Paramètres

— **subsets** (*dict*) – A dictionary where keys are subset indices and values are lists of media file names.

— **formatted_results** (*list*) – A list of dictionaries containing formatted results from the database.

Renvoie

A dictionary where keys are media file names and values are their corresponding chunk texts.

Type renvoyé

transcriptions (dict)

Chatbot.DBEmbed.**get_formatted_results**(*results*)

Connects to the database and retrieves formatted results for the query text.

Paramètres

results (*list*) – A list of dictionaries containing formatted results from the database.

Renvoie

A list of embeddings corresponding to the results. **files** (*list*) : A list of media file names corresponding to the results.

Type renvoyé

embeddings (list)

Chatbot.DBEmbed.**get_media_name**(*folder_name*)

Retrieves the media file name from the S3 bucket based on the folder name.

Paramètres

folder_name (*str*) – The name of the folder to search in the S3 bucket.

Renvoie

The media file name if found, otherwise None.

Type renvoyé

str

Chatbot.DBEmbed.**get_results**(*query_text, model, milvus_collection, collection*)

Searches for the query text in the Milvus collection and returns formatted results.

Paramètres

— **query_text** (*str*) – The text to search for in the Milvus collection.

— **model** (*SentenceTransformer*) – The SentenceTransformer model used for encoding the query text.

— **milvus_collection** (*Collection*) – The Milvus collection to search in.

— **collection** (*Collection*) – The MongoDB collection to retrieve metadata from.

Renvoie

A list of dictionaries containing formatted results from the database.

Type renvoyé

formatted_results (list)

Chatbot.DBEmbed.**write_subsets**(*subsets, transcriptions*)

Writes the subsets to files in a directory structure.

Paramètres

subsets (*dict*) – A dictionary where keys are subset indices and values are lists of media file names.

3.3 Synthesis module

This module is designed to partition large audio files into smaller parts for easier processing and transcription. It generates a synthesis of the complete transcript after processing all parts. This code is part of a larger project that includes audio transcription, synthesis, database management, and a chatbot.

Synthesis.Synthesis.**clean_and_setup_directories**()

Removes and recreates necessary directories.

Synthesis.Synthesis.**consolidate_transcripts_to_docs**(*transcripts_base_dir : str = './transcripts', docs_output_dir : str = './Docs'*)

Consolidates individual transcript files into single document files per media.

Paramètres

- **transcripts_base_dir** (*str*) – Directory containing media-specific transcript folders.
- **docs_output_dir** (*str*) – Directory to save consolidated documents.

Synthesis.Synthesis.**generate_all_syntheses_for_docs**(*docs_dir : str = './Docs', synthesis_output_dir : str = './Synthesis'*)

Generates syntheses (small, medium, large) for all documents in the Docs directory and saves them in the Synthesis directory.

Synthesis.Synthesis.**get_mongo_collection**()

Initializes and returns the MongoDB collection. The connection is established only once (lazy initialization).

Lève

ConnectionError – If unable to connect to MongoDB.

Renvoie

The MongoDB collection object.

Type renvoyé

pymongo.collection.Collection

Synthesis.Synthesis.**large_synthesis**(*transcript : str*) → *str*

Generates a detailed summary of the conference transcript.

Paramètres

transcript (*str*) – The transcript of the conference.

Renvoie

A detailed summary of the conference.

Type renvoyé

str

Synthesis.Synthesis.**medium_synthesis**(*transcript : str*) → *str*

Generates a concise summary of the conference transcript, no longer than 10 sentences.

Paramètres

transcript (*str*) – The transcript of the conference.

Renvoie

A concise summary of the conference.

Type renvoyé

str

`Synthesis.Synthesis.retrieve_transcriptions_from_db()`

Retrieves transcription chunks from MongoDB and organizes them.

Renvoie

A tuple containing (`transcriptions_dict`, `vectors_dict`)

where `transcriptions_dict` maps `media_folder_name` to a list of chunk texts, and `vectors_dict` is currently empty (adjust if vectors are used).

Type renvoyé

tuple

`Synthesis.Synthesis.small_synthesis(transcript : str) → str`

Generates a concise summary of the conference transcript, no longer than 5 sentences.

Paramètres

transcript (*str*) – The transcript of the conference.

Renvoie

A concise summary of the conference.

Type renvoyé

str

`Synthesis.Synthesis.write_transcripts_to_files(transcriptions_data : dict, output_base_dir : str = './transcripts')`

Writes retrieved transcription chunks to individual files organized by media folder.

Paramètres

— **transcriptions_data** (*dict*) – Dictionary mapping `media_folder_name` to list of chunk texts.

— **output_base_dir** (*str*) – Base directory to save the transcripts.

3.4 Evolution module

This module is designed to partition large audio files into smaller parts for easier processing and transcription.

It generates a synthesis of the complete transcript after processing all parts.

This code is part of a larger project that includes audio transcription, synthesis, database management, and a chatbot.

`Evolution.Evolution consolidate_transcripts(parts_dir : str) → str`

Consolidates all transcript files from a directory into a single string.

Paramètres

parts_dir (*str*) – The directory containing the transcript files.

Renvoie

The concatenated content of all transcript files.

Type renvoyé

str

`Evolution.Evolution.generate_summary_and_save(full_transcript : str, output_file_path : str)`

Generates a summary using large_synthesis and saves it to a file.

Paramètres

— **full_transcript** (*str*) – The complete consolidated transcript.

— **output_file_path** (*str*) – The path where the summary should be saved.

`Evolution.Evolution.partition_audio(file_path : str, output_dir : str, part_size_mb : int = 20)`

Partitions a large audio file into smaller parts and saves them.

Paramètres

- **file_path** (*str*) – The path to the input audio file.
- **output_dir** (*str*) – The directory to save the audio parts.
- **part_size_mb** (*int*) – The maximum size of each audio part in megabytes.

Evolution.Evolution.**transcribe_audio_parts**(*parts_dir : str, groq_client : Groq*)

Transcribes all audio parts in a given directory using the Groq API.

Paramètres

- **parts_dir** (*str*) – The directory containing the audio parts.
- **groq_client** (*Groq*) – An initialized Groq client instance.

3.5 Partition module

This module is designed to partition large audio files into smaller parts for easier processing and transcription. It generates a synthesis of the complete transcript after processing all parts. This code is part of a larger project that includes audio transcription, synthesis, database management, and a chatbot.

Evolution.Partition.**generate_response**(*doc : str*)

Generates a synthesis of the complete transcript using the large synthesis function. This function uses the `large_synthesis` function from the Synthesis module.

Paramètres

- doc** (*str*) – The complete transcript text.

Renvoie

The synthesized response.

Type renvoyé

str

Evolution.Partition.**partition_audio**(*file_path : str, output_dir : str, part_size_mb : int = 20*)

Partitions a large audio file into smaller parts and saves them.

Paramètres

- **file_path** (*str*) – The path to the input audio file.
- **output_dir** (*str*) – The directory to save the audio parts.
- **part_size_mb** (*int*) – The maximum size of each audio part in megabytes.

Evolution.Partition.**transcribe_audio_parts**(*parts_dir : str, groq_client : Groq*)

Transcribes all audio parts in a given directory using the Groq API.

Paramètres

- **parts_dir** (*str*) – The directory containing the audio parts.
- **groq_client** (*Groq*) – An initialized Groq client instance.

Index des modules Python

C

Chatbot.Chatbot, ??
Chatbot.DBEmbed, ??

d

data_files_selection.script, ??
data_preprocessing, ??
data_preprocessing.script, ??
data_processing.script, ??
data_processing.script_diarization_chunking, ??
data_processing.script_filtrage, ??
data_processing.script_transcription, ??
data_processing.script_vectorisation, ??

e

Evolution.Evolution, ??
Evolution.Partition, ??
experimental.aglorithmic_speaker_data_extractor.script, ??
experimental.algorithmic_text_detection.script, ??
experimental.detection_speakers_par_des_LLMs.script, ??
experimental.inter_files_diarization.construct_bdd_vectors_from_audios, ??
experimental.inter_files_diarization.find_speaker_in_bdd_by_voice, ??
experimental.inter_files_diarization.vectorisation_utils, ??
experimental.vision_model_extractor_of_speaker_data.script, ??

i

interface_graphique_pour_intégration_des_fichiers, ??

m

moteur_de_recherche_interface_web, ??

S

Synthesis.Synthesis, ??

U

utils.file_queueing_for_processing, ??
utils.file_verification_functions, ??

2 Recherches bibliographiques faites en amont des décisions sur des choix techniques du projet

2.1 Technologies de reconnaissance vocale automatique (ASR)

2.1.1 Introduction

La Reconnaissance Automatique de la Parole (ASR, *Automatic Speech Recognition*) désigne l'ensemble des techniques permettant à une machine de convertir un signal vocal en texte. Bien que perçue aujourd'hui comme une avancée récente de l'intelligence artificielle, la recherche en ASR a commencé il y a plus de cinquante ans.

Les premiers systèmes, dans les années 1960–1970, reposaient sur des méthodes de *reconnaissance par patrons* comme le *Dynamic Time Warping* (DTW) [43], qui comparaient un signal vocal entrant à des gabarits enregistrés. Ces approches étaient limitées à de petits vocabulaires et nécessitaient souvent de prononcer les mots isolément.

Dans les années 1980, l'arrivée des Modèles de Markov Cachés (HMM) [38] a transformé le domaine. Les Coefficients Cepstraux en Fréquence Mel (MFCC) [12] sont devenus la représentation standard du signal, imitant la perception fréquentielle de l'oreille humaine. Les années 1990–2000 ont vu la domination des systèmes hybrides GMM-HMM (HMM avec Modèles de Mélange Gaussien pour les probabilités d'émission), enrichis de modèles de langage n -gram et de lexiques phonétiques étendus [50].

2.1.2 L'approche hybride classique

L'architecture hybride repose sur trois composantes principales : le **modèle acoustique**, le **lexique de prononciation** et le **modèle de langage**.

Le signal vocal est découpé en trames courtes (par exemple 25 ms avec un décalage de 10 ms) supposées quasi stationnaires. Chaque trame est transformée par une analyse spectrale, filtrée selon l'échelle Mel, puis compressée par une transformée en cosinus discrète pour produire des MFCC [12].

Un HMM représente la parole comme une séquence d'états cachés (souvent des parties de phonèmes) avec des probabilités de transition. Les probabilités d'émission, initialement modélisées par des GMM, indiquent la vraisemblance d'observer un vecteur MFCC donné dans un état. L'apprentissage utilise l'algorithme *Baum–Welch* et la reconnaissance le décodage de *Viterbi* [38].

Le lexique phonétique associe à chaque mot une séquence de phonèmes. Le modèle de langage, souvent un modèle n -gram à lissage Kneser–Ney, prédit la probabilité d'une séquence de mots.

Ces composants sont combinés dans un Transducteur à États Finis Pondéré (WFST), souvent noté $H \circ C \circ L \circ G$, qui fusionne le modèle acoustique, le lexique et le modèle de langage. Un algorithme de *beam search* explore efficacement ce graphe pour trouver la transcription la plus probable.

Un avantage de cette approche est le fait qu'elle soit modulaire, interprétable et adaptable : on peut changer le modèle de langage, adapter le lexique ou affiner le modèle acoustique sans reconstruire l'ensemble du système.

2.1.3 L'arrivée des modèles neuronaux

Vers 2010, les réseaux neuronaux profonds (DNN) remplacent progressivement les GMM dans les HMM. Ces systèmes *DNN-HMM hybrides* conservent la structure HMM mais bénéficient d'une meilleure modélisation non linéaire entre les caractéristiques acoustiques et les états phonétiques, ce qui améliore nettement la précision.

À partir de 2014, une nouvelle génération de systèmes vise à supprimer la structure modulaire et à entraîner un modèle unique de l'audio au texte :

Le Connectionist Temporal Classification (CTC) [18] introduit un symbole « blanc » et une fonction de *collapse* qui permet d'apprendre sans alignements précis entre audio et transcription.

Des modèles comme *Listen, Attend and Spell* (LAS) [9] utilisent un mécanisme d'attention pour aligner dynamiquement les représentations audio et les sorties textuelles, supprimant le besoin du blanc CTC. Ces modèles sont performants mais souvent non adaptés au traitement en flux (*streaming*).

Le Recurrent Neural Network Transducer (RNN-T) [17] combine les idées de CTC et d'encodeur-décodeur pour permettre une reconnaissance continue et à faible latence, ce qui en fait un choix courant pour les assistants vocaux en temps réel.

Les Conformers [21] combinent convolution locale et auto-attention pour capturer à la fois les dépendances locales et globales. La technique SpecAugment [36] masque aléatoirement des bandes temporelles ou fréquentielles dans le spectrogramme pour améliorer la robustesse.

Les modèles comme wav2vec 2.0 [4] et HuBERT [24] apprennent à partir de grandes quantités de parole non étiquetée en prédisant des segments masqués ou quantifiés de leur entrée. Après pré-entraînement, une petite quantité de données annotées suffit pour obtenir de bonnes performances, y compris pour des langues peu dotées.

2.1.4 Comparaison hybride et *end-to-end*

Les systèmes hybrides, de type HMM-GMM ou HMM-DNN, conservent une structure *modulaire* où chaque composant — modèle acoustique, lexique et modèle de langage — peut être optimisé séparément. Cette modularité les rend particulièrement adaptés aux environnements à faibles ressources : il est possible, par exemple, de remplacer uniquement le modèle de langage pour adapter le système à un nouveau domaine sans réentraîner l'ensemble. Leur interprétabilité est également un atout dans des contextes où l'explicabilité est cruciale, comme le traitement de documents juridiques ou médicaux. Cependant, cette architecture exige une forte ingénierie manuelle, notamment pour la construction des lexiques et la préparation des données acoustiques et phonétiques.

En termes de performances, les systèmes hybrides peuvent être plus rapides lors du décodage, surtout lorsque le vocabulaire est limité et que l'infrastructure de décodage WFST est optimisée. Toutefois, leur précision plafonne souvent lorsque la quantité de données d'entraînement augmente, et ils peinent à rivaliser avec les architectures modernes sur des corpus très vastes.

À l'inverse, les systèmes *end-to-end* (CTC, RNN-T, LAS, Conformer) traitent l'ensemble de la tâche dans un réseau unique, ce qui simplifie considérablement la conception du système et élimine la phase de construction manuelle du lexique. Sur de grands volumes de données, ils atteignent ou dépassent l'état de l'art en précision, avec des Taux d'Erreur de Mots (WER) inférieurs à 5% sur certains corpus anglophones de référence [21, 5]. Cette progression a été rendue possible par la transition des approches hybrides HMM-DNN [23] vers les architectures *end-to-end* [19]. Toutefois, leur vitesse d'inférence peut varier : les modèles à attention comme LAS peuvent être plus lents en décodage à cause du mécanisme d'alignement dynamique, tandis que les RNN-T et certains Conformers optimisés sont compétitifs pour des applications temps réel.

En termes d'utilisation, les hybrides restent privilégiés dans les environnements contraints (petits appareils, vocabulaire spécifique, langues peu dotées), où leur capacité à intégrer facilement un modèle de langage externe et à fonctionner avec peu de données annotées est décisive. Les modèles *end-to-end*, eux, sont la norme pour les systèmes grand public (assistants vocaux, transcription vidéo, sous-titrage automatique), lorsque les ressources matérielles et les données d'entraînement sont abondantes. Dans des contextes spécialisés, il est courant d'hybrider les deux approches, par exemple en intégrant un modèle de langage externe à un système *end-to-end* pour améliorer la robustesse aux noms propres ou au jargon technique.

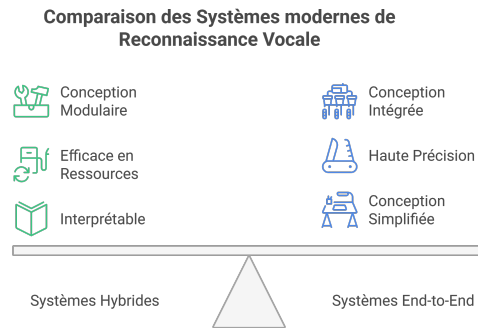


Fig. A.1 Graphique de comparaison entre le système hybride et le système end-to-end pour la reconnaissance audio automatisé

2.2 Modèles de transcription

2.2.1 Comparaisons

NVIDIA Parakeet

La famille *Parakeet* proposée par NVIDIA repose sur des variantes du *FastConformer*, optimisées pour réduire la latence et le coût de calcul [33]. Les modèles incluent des architectures RNNT (Recurrent Neural Network Transducer) et TDT (Two-pass Decoder Transducer) adaptées à différents scénarios. La version *Parakeet-TDT-0.6B-v2* obtient un *Average WER* d'environ 6.05% sur le leaderboard Hugging Face [29], avec une performance remarquable sur LibriSpeech (*test-clean* : 1.69%, *test-other* : 3.70%) [34].

OpenAI Whisper

Le modèle *Whisper* développé par OpenAI [39] est un encodeur-décodeur entraîné de manière faiblement supervisée sur un corpus web multilingue à très grande échelle. La version *large-v3* améliore la précision de 10% à 20% par rapport à la version *large-v2* sur de nombreuses langues [35], tout en conservant une robustesse notable en transcription *zero-shot* sur des domaines et langues variés.

Meta SeamlessM4T-v2

SeamlessM4T-v2 [41] est un modèle multitâche unifié permettant la transcription, la traduction parole-parole et parole-texte. Sur le benchmark multilingue FLEURS, il atteint une WER moyenne de 18.5% sur 77 langues communes, contre 41.7% pour *Whisper-large-v2*, soit une réduction relative d'environ 56%. Ce résultat illustre ses capacités exceptionnelles sur les langues à faibles ressources.

Google Universal Speech Model (USM)

Le *Universal Speech Model* (USM) de Google [52, 40] a été pré-entraîné sur plus de 12 millions d'heures de données non labellisées couvrant plus de 300 langues. Il établit des performances de pointe sur plusieurs benchmarks multilingues, notamment FLEURS, SpeechStew et CORAAL, et se distingue par sa robustesse aux accents et au bruit.

Comparaison des performances

La Table A.2 présente une synthèse des performances de ces modèles sur différents jeux de données publics.

Modèle	Arch. / Taille	Métrique	Valeur	Jeu
Parakeet-TDT-0.6B-v2	TDT/CTC, 0.6B	Average WER	6.05%	Moyenne multi-jeux [29]
Parakeet-TDT-0.6B-v2	TDT/CTC, 0.6B	WER	1.69 / 3.70%	LibriSpeech clean/other [34]
Whisper-large-v3	Enc-dec, ~1.5B+	WER vs v2	-10 à -20%	Multi-langues [35]
SeamlessM4T-v2	Multitâche	WER	18.5%	FLEURS (77 langues) [41]

Tab. A. Comparaison de quelques modèles SOTA en ASR (2025)

2.2.2 Conclusion

En 2025, NVIDIA Parakeet et OpenAI Whisper-large-v3 dominent le domaine de la transcription en anglais et dans des conditions bruitées, tandis que Meta SeamlessM4T-v2 et Google USM excellent sur le multilingue à large échelle.

Au-delà des performances brutes, le choix d'un modèle doit également prendre en compte plusieurs critères pratiques. Les moyens d'utilisation varient : certains modèles, comme USM ou SeamlessM4T-v2, sont principalement accessibles via des API cloud, offrant une grande échelle de calcul mais soulevant des questions de dépendance technologique et de confidentialité. D'autres, tels que Whisper et Parakeet, sont disponibles en open source, permettant une exécution locale ou embarquée, adaptée aux environnements à ressources limitées et aux exigences de souveraineté des données.

La taille des modèles joue aussi un rôle déterminant : les versions légères (moins de 1 milliard de paramètres) facilitent le déploiement sur matériel embarqué avec faible latence, tandis que les grands modèles (plusieurs milliards de paramètres) maximisent la précision mais requièrent des ressources matérielles importantes. Enfin, les contraintes réglementaires, la sensibilité des données traitées et la capacité à personnaliser ou affiner le modèle (*fine-tuning*) doivent guider le choix final en fonction du contexte d'utilisation.

2.3 Segmentation du texte pour un système RAG

La segmentation de textes, ou *chunking*, est une étape cruciale dans les systèmes de Retrieval-Augmented Generation (RAG). Elle consiste à découper un texte en segments exploitables afin de faciliter l'indexation et la récupération d'informations pertinentes pour améliorer la génération de réponses. La manière dont le texte est segmenté influence directement la précision et la couverture des réponses produites par le système.

On distingue principalement deux grandes familles de segmentation :

1. **Segmentation indépendante du sens** : cette approche se base sur la structure du texte, comme la longueur, le nombre de tokens ou des repères syntaxiques simples. Elle est rapide et facile à mettre en œuvre, mais elle ne prend pas en compte le contenu ou le sens du texte.
2. **Segmentation orientée cohérence sémantique** : cette méthode s'appuie sur le contenu, la thématique et les frontières discursives du texte. Elle produit des segments plus cohérents et pertinents, mais nécessite des calculs plus lourds et des algorithmes plus complexes.

Ainsi, le choix de la méthode de segmentation dépend souvent d'un compromis entre rapidité et précision, selon les besoins du système RAG.

Dans la suite de cette partie, nous présenterons plusieurs méthodes employées dans la littérature pour chacune de ces deux familles, en détaillant leurs principes, leurs avantages et leurs limitations.

2.3.1 Méthodes heuristiques (indépendantes du sens)

TextTiling (Hearst, 1997) [22]

Principe algorithmique

1. Diviser le texte en *spans* (blocs) de taille fixe, par exemple k phrases chacun.
2. Construire un vecteur de fréquence de termes pour chaque span après suppression des mots vides et stemming.
3. Calculer la similarité (cosinus ou chevauchement) entre le bloc gauche et le bloc droit autour de chaque frontière candidate.
4. Identifier les chutes de similarité comme marqueurs potentiels de changement de sujet.
5. Appliquer un seuil ou détecter les pics locaux de discontinuité pour poser les frontières définitives.

Avantages

Simple, rapide et facilement interprétable.

Inconvénients

Sensible au choix de la taille des spans et ne capture pas les synonymies.

C99 (Choi, 2000 / 2004 améliorations) [10]**Principe algorithmique**

1. Représenter chaque phrase comme un vecteur de fréquence de termes.
2. Construire une matrice de similarité complète entre toutes les phrases.
3. Appliquer un *ranking mask* local pour lisser et filtrer la matrice.
4. Détecter les blocs cohésifs par partitionnement linéaire (clustering 1D).
5. Placer les frontières aux zones de transition entre blocs.

Avantages

Capture les structures locales et globales.

Inconvénients

Complexité plus élevée (matrice $n \times n$) et dépendance au paramétrage.

LCSeg et chaînes lexicales (Barzilay & Elhadad 1997 ; Galley et al. 2003) [7], [16]**Principe algorithmique**

1. Construire des *chaînes lexicales* : séquences de mots reliés par répétition ou relation sémantique (WordNet, co-occurrence).
2. Pondérer les chaînes selon leur fréquence et distribution dans le texte.
3. Calculer la cohésion locale comme la somme des poids des chaînes couvrant la fenêtre courante.
4. Détecter les ruptures de cohésion comme frontières de segment.

Avantages

Meilleure robustesse sur textes riches en synonymies.

Inconvénients

Dépendance à la désambiguïsation lexicale et aux ressources.

Modèle statistique d'Utiyama et Isahara (2001) [47]

Principe algorithmique

1. Modéliser la probabilité des mots d'un segment via un modèle de langage unigramme.
2. Calculer la vraisemblance du texte pour une segmentation donnée comme produit des probabilités des segments.
3. Utiliser une programmation dynamique pour trouver la segmentation qui maximise cette vraisemblance.

Avantages

Approche probabiliste globale, indépendante du domaine.

Inconvénients

Modèle lexical simple, sensible à la longueur des segments.

BayesSeg (Eisenstein & Barzilay, 2008) [13]

Principe algorithmique

1. Considérer chaque segment comme généré par un modèle de langage distinct.
2. Placer une distribution a priori sur les paramètres (Dirichlet) et le nombre de segments.
3. Utiliser un échantillonnage de type Gibbs pour inférer simultanément les frontières et les paramètres des modèles.

Avantages

Capture la variation lexicale avec un modèle bayésien robuste.

Inconvénients

Coût computationnel élevé, hyperparamètres à régler.

Segmentation par graphes et *min-cut* (graph-partitioning) [28]

Principe algorithmique

1. Représenter chaque unité (phrase) comme un nœud d'un graphe.
2. Pondérer les arêtes par la similarité lexicale entre phrases.
3. Formuler la segmentation comme une coupe minimale ou *normalized cut* optimisant la cohésion intra-segment et séparabilité inter-segment.

Avantages

Capture la structure globale du texte.

Inconvénients

Coût élevé (algorithmes spectraux) et sensibilité au choix des poids.

2.3.2 Méthodes sémantiques (orientées cohérence)TopicTiling (2012) [42]**Principe algorithmique**

1. Entraîner un modèle LDA pour extraire une distribution de thématiques par phrase.
2. Remplacer les vecteurs lexicaux de TextTiling par ces distributions.
3. Appliquer le calcul de similarité et détection de chutes comme dans TextTiling.

Avantages

Améliore la détection de changements thématiques (synonymies, paraphrases).

Inconvénients

Dépendance à un bon paramétrage du LDA.

Text Segmentation as a Supervised Learning Task (2018) [26]**Principe algorithmique**

1. **Formulation** : segmentation comme étiquetage binaire par phrase (la phrase s_i clôt un segment : oui/non).
2. **Encodage phrase** : pour chaque phrase, encoder les mots avec une BiLSTM (2 couches) puis appliquer un *max-pooling* temporel pour obtenir un embedding de phrase.
3. **Encodage document** : empiler une BiLSTM (2 couches) au niveau des phrases pour modéliser le contexte inter-phrases.

4. **Tête de décision** : couche pleinement connectée + *softmax/sigmoid* par frontière pour produire $p_i = \text{Pr}(\text{fin de segment après } s_i)$.
5. **Apprentissage** : minimisation de l'entropie croisée sur les $n - 1$ frontières, avec embeddings de mots pré-entraînés (word2vec).
6. **Décodage** : seuillage τ (optimisé sur dev) ; on ouvre un nouveau segment quand $p_i > \tau$.

Avantages

Apprentissage supervisé à grande échelle (WIKI-727K) ; hiérarchie mots $\rightarrow$ phrases $\rightarrow$ document ; linéaire en longueur et parallélisable GPU.

Inconvénients

Pas de pré-entraînement Transformer ; dépend à la granularité et au seuillage ; sensible aux longues dépendances au-delà de la fenêtre BiLSTM.

SECTOR : Coherent Topic Segmentation and Classification (2019) [3]

Principe algorithmique

1. **Objectif conjoint** : segmentation + classification thématique des segments.
2. **Latent topic embedding** : apprendre un embedding de sujet variant le long du document via un encodeur séquentiel (LSTM) supervisé par en-têtes/étiquettes (WikiSection).
3. **Projection phrase** : produire un vecteur topique e_k par phrase ; détecter les ruptures par discontinuités de trajectoire dans l'espace latent.
4. **Découpage** : appliquer un algorithme de détection de ruptures (seuil sur distance/cosinus ou heuristiques bi-directionnelles) pour placer les frontières.
5. **Étiquetage** : classifier chaque segment via une tête multi-labels (headings/ontologie).
6. **Entraînement** : supervision faible à partir des sections Wikipédia (anglais/allemand, domaines villes et maladies) avec exploration de variantes d'architectures.

Avantages

Segmentation *et* étiquetage unifiés ; très adapté aux documents structurés (WikiSection) ; l'embedding topique est exploitable pour le *retrieval* RAG.

Inconvénients

Biais domaine (en-têtes Wikipédia) ; dépendance à des labels de section ; LSTM moins scalable que des Transformers long-contexte récents.

Two-Level Transformer with Coherence Modeling / CATS (2020) [46]

Principe algorithmique

1. **Hierarchie** : Transformer au niveau mot $\rightarrow$ embeddings de phrase, puis Transformer au niveau phrase (*two-level*).
2. **Tâches auxiliaires de cohérence** : apprentissage conjoint d'objectifs (p.ex. ordre de phrases, inférence de voisinage) pour injecter de la cohérence discursive.
3. **Prédiction de frontières** : tête de classification binaire par frontière de phrase avec distribution calibrée.
4. **Optimization** : fine-tuning bout-en-bout avec pertes pondérées (segmentation + cohérence).
5. **Décodage** : seuil global ou recherche contrainte (p.ex. nombre attendu de segments).

Avantages

Capte des indices de cohérence au-delà de la similarité lexicale ; robuste au bruit ; compatible avec pré-entraînement Transformer.

Inconvénients

Coût mémoire/temps (double niveau) ; besoin d'objectifs auxiliaires bien choisis ; tuning sensible.

Joint Document Segmentation and Segment Labeling (2020) [6]
Principe algorithmique

1. **Modèle séquence-à-séquence structuré** : segmenter et attribuer une étiquette de segment simultanément.
2. **Encodeur** : représentation contextuelle de phrases (RNN/Transformer) avec traits discursifs.
3. **Décodage structuré** : produire une séquence d'actions ("B" = début de segment, "I" = intérieur, + label), avec contraintes globales.
4. **Apprentissage** : perte jointe (frontières + labels) ; régularisation par cohérence intra-segment.
5. **Inférence** : Viterbi/beam search sous contraintes pour respecter la structure segment/label.

Avantages

Mutualise les signaux segmentation/étiquetage (meilleures frontières) ; pratique pour RAG thématique (indexation immédiate par label).

Inconvénients

Nécessite des labels de segment ; décodage plus coûteux ; sensibilité au déséquilibre des classes.

Text Segmentation by Cross-Segment Attention (2020) [27]

Principe algorithmique

1. **Encodage** : Transformer hiérarchique pour obtenir des embeddings de phrases.
2. **Cross-Segment Attention (CSA)** : pour une frontière candidate, agréger l'attention croisée entre la fenêtre précédente et suivante afin de modeler explicitement la dissimilarité inter-segments.
3. **Tête frontière** : classifier chaque position via les features CSA (p.ex. pooling d'attention, contraste gauche/droite).
4. **Entraînement** : supervision sur frontières vraies avec perte binaire ; régularisation par marge entre scores gauche/droite.
5. **Décodage** : seuillage + post-traitement (espacement minimal, fusion de segments courts).

Avantages

Compare *explicitement* les régions adjacentes ; efficace pour ruptures nettes de sujet ; facilement plug-in sur divers encodeurs.

Inconvénients

Fenêtres et hyperparamètres sensibles ; peut manquer des transitions subtiles ; coût quadratique de l'attention si non bornée.

BERT-based Unsupervised Topic Segmentation for Meetings (2021) [45]

Principe algorithmique

1. **Embeddings contextuels** : encoder les énoncés avec un modèle de type BERT finement adapté au domaine (réunions/transcriptions).
2. **Signal de rupture** : calculer une distance/similarité (cosinus) entre fenêtres contextuelles glissantes avant/après chaque frontière candidate.
3. **Lissage** : filtrage/savitzky-golay ou moyenne mobile pour stabiliser le signal de discontinuité.

4. **Décision** : détecter les pics au-delà d'un seuil adaptatif (p. ex. par maximums locaux) comme frontières.
5. **Optionnel** : affiner via clustering des segments ou *change point detection* bayésien.

Avantages

Aucune annotation nécessaire ; exploite des embeddings profonds ; très simple à intégrer en pré-processing RAG.

Inconvénients

Heuristique (pas d'objectif supervisé direct) ; seuils sensibles ; qualité dépendante du modèle d'embedding et du domaine.

Improving Long-Document Topic Segmentation with Enhanced Coherence (2023) [51]

Principe algorithmique

1. **Encodeur long-contexte** : utiliser Longformer/Transformer sparse pour encoder des documents entiers.
2. **Tâches de cohérence** : (i) *Topic-Shift Sentence Prediction (TSSP)* — prédire si une phrase marque un changement de sujet ; (ii) *Contrastive Self-Supervised Learning (CSSL)* — rapprocher des phrases du même segment et éloigner celles de segments différents.
3. **Tête de segmentation** : classification binaire des frontières, informée par les représentations enrichies (TSSP+CSSL).
4. **Entraînement** : apprentissage multi-tâches (pertes supervisées + contrastives) sur WIKI-727K et WikiSection ; *fine-tuning* domain-adapté.
5. **Décodage** : seuillage + contrainte de longueur minimale de segment ; possibilité d'ajuster le nombre de segments attendu.

Avantages

Excellente couverture contextuelle (long documents) ; objectifs de cohérence très pertinents pour transitions de sujet ; forte transférabilité inter-domaines.

Inconvénients

Coûts mémoire/temps plus élevés ; tuning non trivial des objectifs contrastifs ; dépendance aux données Wikipédia pour la supervision.

LLM-based Prompted Segmentation (2025) [49, 51]

Principe algorithmique

1. **Zéro-shot / peu d'exemples** : formuler un *prompt* qui instruit un LLM (p. ex. GPT-4-class) à *marquer explicitement* les frontières de segments donnés un texte.
2. **Découpage hiérarchique** : itérer (coarse→fine) — d'abord thèmes majeurs, puis sous-segments, en conservant des marqueurs normalisés (<SEG>).
3. **Validation automatique** : post-vérifier par similarité cosinus des embeddings (p. ex. du LLM ou d'un encodeur bi-encodeur) entre segments adjacents pour confirmer les ruptures.
4. **Auto-labeling optionnel** : demander au LLM d'assigner des titres de segment (utiles pour l'index RAG) puis contrôler qualité via règles (longueur min., chevauchement).
5. **Boucle d'alignement RAG** : évaluer la segmentation par feedback de récupération (MRR/F1 des segments pertinents) et raffiner le prompt (ou insérer quelques exemples).

Avantages

Aucune donnée annotée ; très flexible (granularité ajustable) ; génération simultanée de résumés/titres utilisables en RAG.

Inconvénients

Variabilité/instabilité des frontières ; coût ; besoin de garde-fous automatiques (validation par embeddings, contraintes de longueur).

2.4 Introduction

La diarisation automatique du locuteur est un domaine en évolution rapide, notamment grâce aux approches basées sur l'apprentissage profond. Trois familles de modèles dominent actuellement la littérature et les applications pratiques : **pyannote.audio**, **NVIDIA NeMo** et les modèles End-to-End Neural Diarization (EEND) récents, dont **DiaPer**. Chacune de ces approches propose des solutions différentes aux défis centraux de la diarisation, en particulier la détection des locuteurs en chevauchement, la généralisation à des environnements variés et la possibilité d'un traitement en ligne.

3 Modèles récents de diarisation

3.1 pyannote.audio

pyannote.audio (v3.x) est l'une des solutions les plus utilisées dans la communauté académique et industrielle. Elle repose sur une pipeline classique (détection vocale, segmentation, extraction d'embeddings, regroupement), mais optimisée avec des modules récents, notamment un modèle de segmentation par powerset qui gère efficacement le chevauchement. Les benchmarks publics montrent des performances solides, par exemple un DER de 22,2% sur AMI-SDM et 21,7% sur DIHARD III, tout en restant facilement utilisable grâce à son intégration avec Hugging Face [14].

3.2 NVIDIA NeMo

De son côté, NVIDIA NeMo met l’accent sur des solutions de diarisation adaptées à la production. En plus de pipelines basées sur le regroupement et la réassignation (VBx), NVIDIA propose le Multi-Scale Diarization Decoder (MSDD) qui améliore nettement la gestion du chevauchement. Plus récemment, des modèles streaming tels que Sortformer permettent une diarisation en temps réel avec des latences inférieures à la seconde. Le modèle Sortformer atteint par exemple un DER de 4,66% sur AMI-SDM avec une latence de 600 ms [32, 31].

3.3 EEND et DiaPer

Enfin, les approches End-to-End Neural Diarization (EEND) visent à remplacer la pipeline traditionnelle par un traitement entièrement neuronal, dans lequel un réseau prédit directement les assignations frame × locuteur. Le modèle DiaPer (2024) illustre cette évolution en exploitant une architecture basée sur Perceiver pour estimer les attracteurs de locuteurs de manière non-autoregressive. Cette approche excelle dans les situations de chevauchement massif et montre des gains notables par rapport à EEND-EDA sur CALLHOME et d’autres corpus large-bande [30].

3.4 Comparaison des approches

Le tableau A.2 compare ces trois approches selon plusieurs critères clés.

Critère	pyannote.audio v3.x	NVIDIA (MSDD/Sortformer)	NeMo	EEND moderne (DiaPer)
Type	Pipeline cascadée optimisée	Cascadée enrichie + streaming	+	End-to-end (Perceiver)
Chevauchement	Très bon (powerset v3)	Excellent (MSDD)		Naturellement performant
Online/Streaming	Plutôt offline	Oui (<1 s latence)		Principalement offline
Facilité d’usage	Très simple (HF)	Bonne, NeMo	nécessite	Requiert entraînement
Généralisation	Excellente multi-domaine	Excellente en production		Dépend du corpus
Scores publics	AMI-SDM : 22.2; DIHARD III : 21.7; VoxConverse : 11.3	AMI-SDM : 4.66 (streaming)		Sup. à EEND-EDA sur CALLHOME
Licence/Code	Open-source (HF/GitHub)	Open-source (NeMo)		Open-source

Tab. A. Comparaison des trois principales approches de diarisation.

3.5 Conclusion

Ces résultats suggèrent que le choix du modèle doit se faire en fonction du contexte d’application. Pour un **déploiement en production avec contraintes de latence**, NeMo

est particulièrement adapté. Pour un usage **robuste et flexible en recherche ou prototypage**, `pyannotate.audio` représente une option privilégiée. Enfin, dans les cas de **fort chevauchement** ou lorsque l'on souhaite explorer des architectures end-to-end, les modèles EEND tels que DiaPer offrent des perspectives prometteuses.

Abstract

Ce rapport présente les travaux réalisés dans le cadre de mon stage de fin d'études au sein de CS Group, consacrés à la valorisation des fichiers média de conférences. L'objectif principal était de concevoir et de mettre en place une chaîne de traitement permettant une extraction fluide et simplifiée des données. Pour cela, j'ai mobilisé des méthodes issues de l'intelligence artificielle et de la science des données, en m'appuyant sur des outils tels que Python, Docker ou encore AWS. Les résultats ont permis de démontrer, à travers une preuve de concept, le potentiel d'un tel système dans un contexte industriel et d'ouvrir la réflexion vers de nouvelles pistes d'évolution en intelligence artificielle. Ce stage m'a également offert l'opportunité de consolider mes compétences en systèmes de RAG, tout en me sensibilisant aux enjeux spécifiques liés au traitement et aux flux de données, ainsi qu'à l'importance croissante de la science des données dans le domaine de l'aviation.

This report presents the work carried out during my final-year internship at CS Group, focused on the enhancement of conference media files. The main objective was to design and implement a processing pipeline enabling smooth and simplified data extraction. To achieve this, I applied methods from artificial intelligence and data science, relying on tools such as Python, Docker, and AWS. The results demonstrated, through a proof of concept, the potential of such a system in an industrial context and opened the way to new directions for future developments in artificial intelligence. This internship also allowed me to strengthen my skills in RAG systems while gaining insight into the specific challenges related to data processing and flows, as well as the growing importance of data science in the field of aviation.

Keywords RAG, NLP, ASR, Speech Processing

Je soussigné, EL MANNOUY Ayman, atteste que le présent travail est le fruit de mes propres recherches, réflexions et analyses. Aucune partie de ce rapport n'a été recopiée ou reproduite sans référence explicite à la source utilisée. Toutes les citations, données et références empruntées à d'autres auteurs ou documents sont dûment mentionnées conformément aux usages académiques.

Je confirme ainsi que ce travail est original et qu'il ne comporte aucune forme de plagiat.

Fait à Toulouse, le 26/08/2025

Signature :

A handwritten signature in black ink, consisting of a stylized 'A' shape with a horizontal line extending to the right and a vertical line intersecting it.